

Description and Basic Inference with R

Jinsong Chen

Table of contents

1	Read and Prepare Observations	3
1.1	R, RStudio, a script, and Quarto	3
1.2	A data frame is a table of cases and variables	4
1.3	Inspect, select, and derive a variable	5
1.4	Recorded cases and a proposed population	7
2	Describe Distributions and Groups	9
2.1	Shape, centre, and spread	9
2.2	Group summaries and weighted means	12
2.3	Counts, proportions, and a binary indicator	14
2.4	Transformations and the Normal model	15
2.5	A compact distribution catalogue in R	17
3	Sampling Variation and Standard Errors	21
3.1	A target, an estimator, and a realized estimate	21
3.2	The first repeated-sample calculation	23
3.3	Bias and precision	26
3.4	SE follows the unit and the comparison	27
3.5	Concentration and sampling shape	30
4	Intervals and Tests for Means	31
4.1	Estimate, SE, critical value, and interval	31
4.2	Supplied summaries and a planned direction	34
4.3	Paired observations and complete pairs	35
4.4	Two independent groups and Welch's method	38
4.5	Coverage belongs to the procedure	40
5	Correlations and Their Inference	41
5.1	Begin with the plotted observations	41
5.2	Covariance and correlation matrices	45
5.3	Pearson, rank, and point-biserial coefficients	46
5.4	A selected correlation test	47
5.5	One-control partial correlation	49

6	Report the Answer	51
6.1	A result connects numbers to a question	51
6.2	Connect a report to its source	51
7	Reading Bridge	52
	Key terms	52
	Further reading	53
	Exercises	54
	Exercise 1: Count the right observations (Foundational)	54
	Exercise 2: Center, spread and a changed observation (Foundational)	54
	Exercise 3: A missing score is not zero (Foundational)	54
	Exercise 4: Choose the group denominator (Foundational)	54
	Exercise 5: Read a relationship before evaluating it (Integrative)	55
	Exercise 6: Report paired differences (Integrative)	55
	Exercise 7: Simulate a sampling distribution and read its SE (Integrative)	55
	Exercise 8: Name the estimand, estimator, and estimate, then calculate the SE (Integrative)	55
	Exercise 9: An interval and its question (Foundational)	56
	Exercise 10: A selected relationship (Integrative)	56
	References	56

Chapter overview

This chapter makes the descriptions of Chapter 2 and the mean inference of Chapter 3 executable in R. We read and select observations, describe distributions and groups, and simulate the variation of an estimator. We then connect an estimate, its standard error, a reference distribution and a critical value to an interval and a test. Relationships receive the same treatment: a graph and covariance lead to correlation matrices, selected correlation tests and one-control linear adjustment. Throughout, the calculation stays beside the observations, conditions and question that give it meaning. A short Quarto report connects the final answer to the code that produced it.

Three students scored 50, 60 and 70 points. Their mean is $(50 + 60 + 70)/3 = 60.0000$ points. R gives the same result by applying a function to a vector:

```
scores <- c(50, 60, 70)
mean(scores)
#> [1] 60
```

`c()` collects the numbers into a **vector**, an ordered collection of values. The assignment symbol `<-` gives the vector the name `scores`. `mean(scores)` supplies it to the function `mean()`, which returns 60. Naming the observations lets later commands use the same values without retyping them. This answer describes the three recorded scores directly. Using it to estimate another group's mean would add a target and an account of how these observations inform that target.

The chapter assumes no previous R course. Familiar browser examples from Chapters 3–4 provide a starting point; a saved script now lets us run a connected analysis again. The Computer Lab gives short setup checks and practice using the same calculations.

1 Read and Prepare Observations

1.1 R, RStudio, a script, and Quarto

R performs statistical calculations and draws graphs (R Core Team, 2025). An **object** is a named value, vector, table or result in its session; a **function** performs an operation on inputs. **RStudio** provides an editor and Console for writing and running R commands. Save the opening commands in a `.R` script: the script records how an answer was obtained, and running it in order recreates the objects in a fresh session.

Quarto combines prose, mathematics, executable code, tables, figures and references in a `.qmd` source and renders a report (Posit Software, PBC, 2026). Section 6 shows what that connection accomplishes. The Computer Lab's setup reference covers installation, launch and Help; its final round uses a prepared report rather than requiring a new document from scratch.

All statistical calculations and plots here use **base R**, including the stats, graphics, utils and datasets packages supplied with R. A **package** collects functions and data. Additional report support, such as knitr and rmarkdown, lets Quarto execute R; its installation belongs with the reporting example. Chapter 6 introduces a small additional plotting vocabulary after this base-R route.

Square brackets select positions or values. R positions start at 1; a comparison produces a logical vector of TRUE and FALSE values:

```
options(digits = 10, width = 65)
length(scores)
#> [1] 3
scores[2]
#> [1] 60
scores >= 60
#> [1] FALSE TRUE TRUE
scores[scores >= 60]
#> [1] 60 70
mean(x = scores[scores >= 60])
#> [1] 65
```

The selected scores are 60 and 70, whose mean is 65.0000. Selection leaves `scores` unchanged. In the last call, `x` names an **argument**, an input or option supplied to a function. Inside a call, `=` supplies an argument; `<-` assigns an object. This computing meaning of argument differs from a study argument. R is case-sensitive, so `scores` and `Scores` name different objects. The Console marker `[1]` indicates the first displayed position, rather than being part of the answer. The `digits` setting changes printed precision, not stored values.

1.2 A data frame is a table of cases and variables

A **data frame** is a table whose columns can hold different types of values. In Chapter 2's data-matrix notation, x_{ij} is case i 's value on variable j ; the arrangement also accommodates text and factors. Its layout should make the observational unit clear. In the next table, a row represents one student, while the columns give a student identifier, a programme code, a baseline score, and a follow-up score. Student B's follow-up score is unavailable. Chapter 2 explained how IDs, long and wide layouts, and calculation denominators represent the same observations. Here R carries out those operations. A row can instead record one person on one occasion or under one condition; count the represented units as well as the rows.

```
small <- data.frame(
  student = c("A", "B", "C"), program_code = c(1, 2, 2),
  baseline = c(50, 60, 70), followup = c(55, NA, 78))
```

```
small
#>   student program_code baseline followup
#> 1      A             1      50      55
#> 2      B             2      60      NA
#> 3      C             2      70      78
```

The `$` operator selects a named column. Thus `small$followup` is the vector containing 55, NA, and 78. An ordinary `mean()` of that vector returns NA, signalling that a value needed for the calculation is unavailable. Setting `na.rm = TRUE` calculates the mean of the recorded values. We count those same values beside the mean:

```
mean(small$followup, na.rm = TRUE)
#> [1] 66.5
sum(!is.na(small$followup))
#> [1] 2
```

`is.na()` asks whether each entry is missing; `!` reverses the logical answers. `sum()` counts the TRUE values as ones, so the second line gives 2. The available follow-up mean is $(55 + 78)/2 = 66.5000$ points. Treating the missing score as zero would answer a different question using an invented measurement. Excluding it from the calculation makes the denominator explicit, without recovering the score or establishing that the included students represent anyone else.

For change, subtract the baseline score from the follow-up score within each student:

```
paired_rows <- complete.cases(small[c("baseline", "followup")])
small$change <- small$followup - small$baseline
mean(small$change[paired_rows])
#> [1] 6.5
```

`complete.cases()` identifies the students with both measurements. The changes are 5, NA, and 8 points, giving a complete-pair mean of $(5 + 8)/2 = 6.5000$. Subtracting the two separate available means happens to give $66.5000 - 60 = 6.5000$ too. However, the baseline mean uses three students and the follow-up mean uses two. Their numerical agreement here does not establish agreement of the student sets. A paired calculation keeps the same students in both measurements, a principle we return to in Section 3.4.

1.3 Inspect, select, and derive a variable

The `airquality` documentation describes daily New York measurements from May to September 1973. A row is a day. `Ozone` is ozone in parts per billion (ppb), `Wind` wind speed in miles per hour, `Temp` maximum temperature in degrees Fahrenheit, and `Solar.R` solar radiation in

Langleys. `Month` and `Day` identify dates within that period. Inspect the table before choosing a calculation:

```
air <- airquality
head(air, 2)
#>   Ozone Solar.R Wind Temp Month Day
#> 1    41     190  7.4   67     5   1
#> 2    36     118  8.0   72     5   2
dim(air)
#> [1] 153   6
c(rows = nrow(air), recorded = sum(!is.na(air$Ozone)),
  missing = sum(is.na(air$Ozone)))
#>    rows recorded missing
#>    153     116     37
ozone <- air$Ozone[!is.na(air$Ozone)]
```

`head()` shows the first rows; `dim()` returns 153 rows and six columns. `is.na()` identifies unavailable values and `!` reverses those logical answers. `sum()` counts TRUE as one, giving 116 recorded ozone values and 37 missing values. The mean of `ozone` will therefore describe 116 days. A percentage of days with measurements would instead use all 153 as its denominator: $100(116/153) = 75.8170\%$.

Data-frame indexing uses `[rows, columns]`. Leaving the column position empty keeps every column from the chosen rows; `&` requires both rules:

```
july <- air[air$Month == 7 & !is.na(air$Ozone), ]
c(n_July = nrow(july), mean_July = mean(july$Ozone))
#>    n_July mean_July
#> 26.00000000 59.11538462
air$temp_C <- (air$Temp - 32) * 5 / 9
small$program <- factor(small$program_code, levels = c(1, 2),
  labels = c("Standard", "Practice"))
small[c("student", "program_code", "program")]
#>   student program_code  program
#> 1      A             1 Standard
#> 2      B             2 Practice
#> 3      C             2 Practice
```

July contributes 26 recorded days with mean 59.1154 ppb. This restriction selects a different set of days. Converting temperature to Celsius keeps the observations and changes their units. The programme codes 1 and 2 name categories: their numerical storage does not make an average code a meaningful average programme. A **factor** stores these categories with named levels and an explicit order.

A relationship requires jointly recorded measurements. For ozone and solar radiation, only the rows that record both values enter the calculation.

```
#> [1] 111
```

There are 111 complete pairs. A missing value in an unrelated column does not remove a pair under this rule. Complete records define the calculation's cases; they do not establish random sampling. Likewise, NA represents an unavailable value, whereas zero is a recorded value. Chapter 2's zero-grade example shows why a measurement definition and codebook must support any decision to convert zeros to missing values.

1.4 Recorded cases and a proposed population

The 153 days form a fixed historical record for a particular city and period. An observed mean can describe its recorded measurements fully. A claim about the 37 days with unavailable ozone, other years, or other cities needs an additional target and a justification connecting the available days to it.

For example, a question about the mean recorded ozone in this dataset can be answered directly. A question about the mean ozone of all New York summer days treats these observations as evidence about a wider population. The arithmetic might begin with the same mean, but the second use adds assumptions about selection, time, and measurement. Section 3 makes those assumptions visible when moving from an observed SD to the modeled uncertainty of a mean.

Chapter 2 also distinguished selection schemes. They describe how units enter a study; filtering recorded values in R describes which observations enter a particular calculation. A complete-case filter supplies no random sampling mechanism. The invented school system gives a compact reminder:

Table 1*Recognize the sampling procedure before calculating.*

Scheme	Selection from the school system
Simple random sampling	Every set of 100 students from the register has the same chance.
Systematic sampling	Choose a random start, then every 200th student; the ordering of the list matters.
Stratified random sampling	Select a probability sample within every school-type stratum.
Cluster sampling	Randomly select schools and include their eligible students.
Multistage sampling	Randomly select schools, then students within those schools.

Probability sampling gives eligible units known nonzero inclusion probabilities, which need not be equal. An open invitation, convenience recruitment, or quotas alone does not supply those probabilities. Frame coverage and nonresponse still matter after probability selection. Sampling strata concern selection; Chapter 3's school blocks concern allocation of conditions. Cluster sampling and assignment of clusters are different design decisions, even when both produce records grouped within schools.

We can carry out the simple random selection with a register of IDs. For the invented population of 20,000 students, select 100 without replacement:

```
set.seed(60508)
selected_id <- sample.int(20000, 100, replace = FALSE)
head(selected_id)
#> [1] 2507 12971 506 15796 10207 17912
c(selected = length(selected_id), distinct = length(unique(selected_id)))
#> selected distinct
#>      100      100
```

`sample.int()` selects from the integers 1 through 20,000. Setting `replace = FALSE` prevents selecting an ID twice in this sample. The selected count is 100; each student has inclusion probability $100/20,000 = .0050$. A new repetition starts from the full register, so students may appear again in a different sample. The IDs are identifiers, not grades: without a population score vector this code cannot reproduce Chapter 2's illustrative means of 10.6, 11.2, and 10.9. Section 3 changes to generating measurements from a stated probability model. General unequal-probability estimation and the SEs for all survey designs extend beyond the methods developed here.

2 Describe Distributions and Groups

2.1 Shape, centre, and spread

A distribution shows how the values are arranged. Before reducing the 116 ozone measurements to numbers, inspect their shape. A histogram groups nearby values into intervals and counts the observations in each interval; a boxplot compresses their location and spread.

```
oldpar <- par(mfrow = c(1, 2), mar = c(4, 4, 1, .5))
hist(ozone, breaks = seq(0, 180, by = 20), col = "grey85",
     xlab = "Ozone (ppb)", ylab = "Recorded days", main = "")
boxplot(ozone, horizontal = TRUE, xlab = "Ozone (ppb)")
par(oldpar)
```

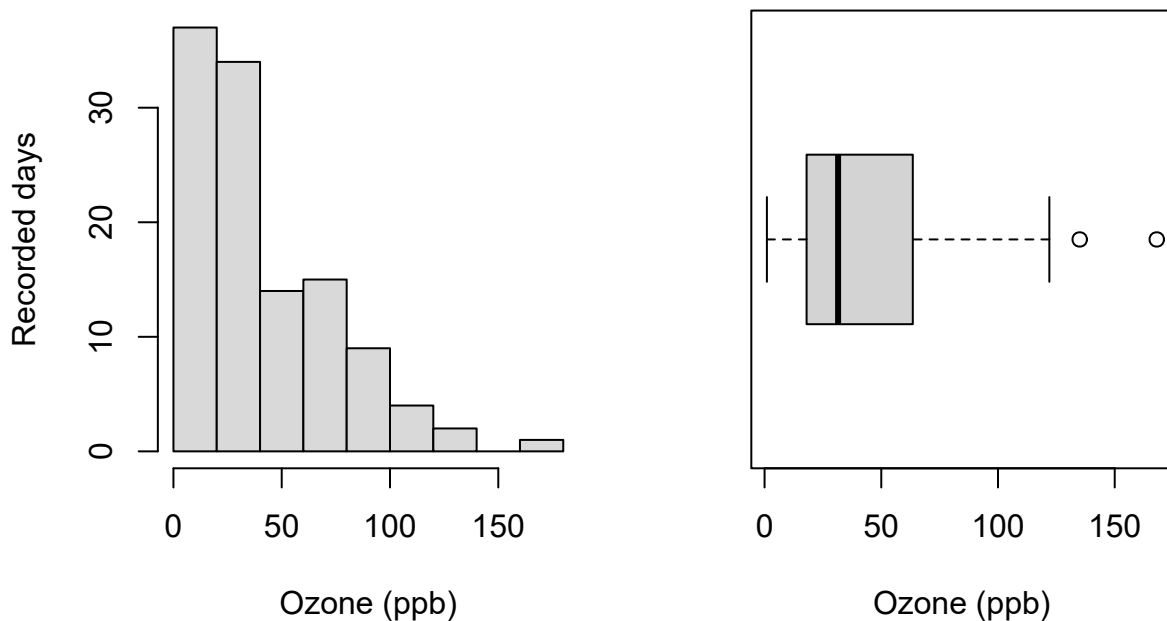


Figure 1: Recorded ozone values on 116 days, displayed as a histogram and boxplot.

The **breaks** vector specifies the histogram boundaries. Changing those boundaries can reveal or conceal detail while leaving the observations unchanged. Most days have lower values, with a right tail reaching 168 ppb. The boxplot marks the median and a central box between its hinges. Each whisker reaches the most distant value within 1.5 box lengths of the corresponding hinge. Points beyond whiskers invite examination; the display alone does not justify removing them. Boxplot hinges can differ from the numerical quartile convention used below. Chapter 2

used the mean and median to describe location, and variance, SD, and IQR to describe spread. Their calculations preserve different information. For observations $x_1, \dots, x_n$,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1},$$

$$s = \sqrt{s^2}, \quad IQR = Q_3 - Q_1.$$

The summation sign $\sum$ means to add the indicated terms. The variance first measures each value's deviation from the mean, squares those deviations, adds them, and divides by $n-1$. Squaring prevents positive and negative deviations from cancelling, but gives variance squared measurement units. Taking the square root returns the SD to the original units.

For the three quiz scores, the deviations from 60 are -10 , 0 , and 10 . Their squared deviations sum to 200. Thus $s^2 = 200/(3-1) = 100.0000$ squared points and $s = 10.0000$ points. R lets us inspect the sum as well as the final summaries:

```
c(mean = mean(scores), ss = sum((scores - mean(scores))^2),
  var = var(scores), sd = sd(scores))
#> mean    ss   var   sd
#>   60   200   100   10
changed_scores <- c(50, 60, 100)
c(mean = mean(changed_scores), median = median(changed_scores),
  sd = sd(changed_scores))
#>      mean      median      sd
#> 70.00000000 60.00000000 26.45751311
```

Replacing 70 by 100 moves the mean from 60.0000 to 70.0000 and raises the SD to 26.4575. The median remains 60.0000 because it is still the middle ordered value. This illustrates the median's resistance to one extreme value; it does not make either center universally preferable.

Why divide by $n-1$? The deviations around a sample mean sum to zero. Once $n-1$ are known, the last deviation is determined. Under independent sampling from a common distribution with finite variance, this denominator also makes the sample variance unbiased for the process variance. Section 3.3 shows that property through repeated samples. It is a claim about the estimator's average across repetitions, not a guarantee that one sample variance equals the process variance.

For ozone, use the same 116-value vector throughout:

```

c(mean = mean(ozone), median = median(ozone), sd = sd(ozone))
#>      mean      median      sd
#> 42.12931034 31.50000000 32.98788451
quantile(ozone, probs = c(0, .25, .5, .75, 1), type = 7)
#>    0%    25%    50%    75%   100%
#>   1.00  18.00  31.50  63.25 168.00
IQR(ozone, type = 7)
#> [1] 45.25

```

The mean is 42.1293 ppb, median 31.5000 ppb, and SD 32.9879 ppb. The mean exceeds the median, consistent with the long right tail. The first and third quartiles are 18.0000 and 63.2500 ppb, giving IQR 45.2500 ppb; the range is 1–168 ppb. The SD describes spread around the mean, while the IQR describes the middle half of the ordered observations.

Small datasets make quartile conventions especially visible. R's default `type = 7` interpolates between ordered values. For a proportion p , it uses the position $h = 1 + (n - 1)p$. Applied to 1, 3, 3, and 5,

$$Q_1 = 1 + .75(3 - 1) = 2.5000,$$

$$Q_3 = 3 + .25(5 - 3) = 3.5000.$$

The positions are 1.75 and 3.25, respectively, so each calculation moves part of the distance between neighboring values. This gives IQR 1.0000, whereas Chapter 2's medians-of-halves rule gives quartiles 2.0000 and 4.0000, and IQR 2.0000. R's own `quantile(x, c(.25, .75), type = 7)` returns 2.5000 and 3.5000 for these four values, and `IQR(x, type = 7)` returns the matching 1.0000.

```

#> 25% 75%
#> 2.5 3.5
#> [1] 1

```

A reproducible report names the convention when it affects the answer. The different IQRs here come from different rules applied to the same observations.

Consider two invented groups with scores 58, 60, 62 and 40, 60, 80. Both balance at 60, but their distances from that center differ.

```

#> [1] 60 60
#> [1] 2 20

```

Their means are both 60.0000, while their SDs are 2.0000 and 20.0000. A report of equal means would conceal a tenfold difference in SD. This is why a group comparison keeps counts, centers, and spreads together, and why a display often adds information that the means alone cannot provide.

2.2 Group summaries and weighted means

For the ozone records, the month codes identify groups of days. Turn the codes into a factor whose labels appear in calendar order:

```
air_observed <- air[!is.na(air$Ozone), ]
air_observed$month <- factor(air_observed$Month, levels = 5:9,
  labels = c("May", "June", "July", "August", "September"))
cbind(days = table(air_observed$month),
  mean = tapply(ozone, air_observed$month, mean),
  sd = tapply(ozone, air_observed$month, sd))
#>      days      mean      sd
#> May      26 23.61538462 22.22444946
#> June      9 29.44444444 18.20790427
#> July     26 59.11538462 31.63583654
#> August   26 59.96153846 39.68121043
#> September 29 31.44827586 24.14182235
```

5:9 generates the integers from 5 through 9. The `levels` argument states their order, and `labels` gives the corresponding month names. `table()` counts the observations in each group; `tapply()` applies a supplied function, here `mean` or `sd`, to the values within each factor level; `cbind()` places the three results side by side.

The summaries keep each mean beside the number and spread of its recorded values:

Table 2

Recorded ozone by month; means and SDs are in ppb.

Month	Recorded days	Mean ozone	SD ozone
May	26	23.6154	22.2244
June	9	29.4444	18.2079
July	26	59.1154	31.6358
August	26	59.9615	39.6812
September	29	31.4483	24.1418

June's mean describes only nine recorded days. Its count matters when interpreting its position among the other monthly means. The Computer Lab varies the month selection and weighting within this same analysis.

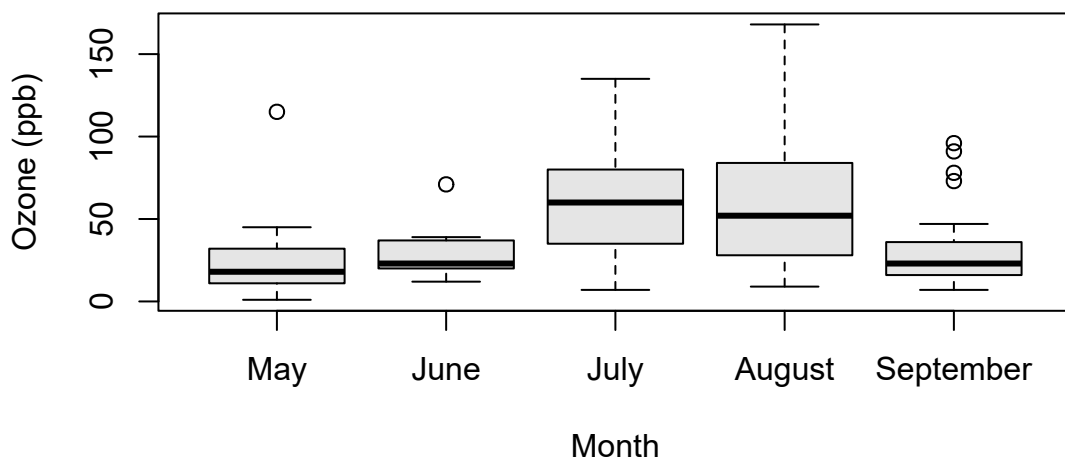


Figure 2: Recorded ozone distributions by month; each box uses that month's available measurements.

The boxes show the higher centers in July and August alongside the within-month variation. The observed monthly differences summarize this record. They do not by themselves isolate an effect of season from other changes during the period.

The **grand mean** over individual observations weights each group mean by its group count:

$$\bar{x} = \frac{\sum_g n_g \bar{x}_g}{\sum_g n_g}.$$

For two observations with mean 20 and eight with mean 50, the grand mean is $(2 \times 20 + 8 \times 50)/10 = 44.0000$. The unweighted average $(20 + 50)/2 = 35.0000$ gives each group equal weight instead. Chapter 2's invented waiting groups make this visible from the observations: A contains 1 and 3 minutes, while B contains 3, 3, 5, and 9. Their means are 2.0000 and 5.0000, their SDs are 1.4142 and 2.8284, and the overall mean is $(2 \times 2 + 4 \times 5)/6 = 4.0000$ minutes.

```
#>      mean_A      mean_B      sd_A      sd_B
#> 2.0000000000 5.000000000 1.414213562 2.828427125
#> [1] 4
```

Combining the two groups gives 4.0000 minutes over the six waits. The same count-weighting rule applied to the five monthly ozone means returns 42.1293 ppb. Giving each month equal weight instead would describe the average month, rather than the average recorded day. For June and July, the two recorded-day counts are 9 and 26:

```
month_n <- tapply(air$Ozone, air$Month, function(x) sum(!is.na(x)))
month_mean <- tapply(air$Ozone, air$Month, mean, na.rm = TRUE)
c(recorded_day_mean = weighted.mean(month_mean[c("6", "7")],
                                     month_n[c("6", "7")]),
  equal_month_mean = mean(month_mean[c("6", "7")]))
#> recorded_day_mean equal_month_mean
#>      51.48571429      44.27991453
```

The recorded-day mean is 51.4857 ppb; the equal-month mean is 44.2799 ppb. Neither weighting rule can be read from the two means alone: the question determines whether days or months receive equal weight.

2.3 Counts, proportions, and a binary indicator

The `mtcars` documentation describes 32 car-model records from the 1974 *Motor Trend* magazine, covering 1973–1974 models. Each row is a car model. The variable `mpg` gives fuel economy in miles per US gallon, while `am` codes automatic transmission as 0 and manual transmission as 1.

Use 20 mpg as a teaching threshold, and label both categories before forming their two-way table:

```
cars <- mtcars
cars$transmission <- factor(cars$am, levels = c(0, 1),
  labels = c("Automatic", "Manual"))
cars$mpg20 <- factor(cars$mpg >= 20, levels = c(FALSE, TRUE),
  labels = c("Below 20", "At least 20"))
tab <- table(cars$transmission, cars$mpg20)
tab
#>
#>      Below 20 At least 20
#> Automatic    15         4
#> Manual       3         10
```

```

100 * prop.table(tab, margin = 1)
#>
#>           Below 20 At least 20
#> Automatic 78.94736842 21.05263158
#> Manual    23.07692308 76.92307692
100 * prop.table(tab, margin = 2)
#>
#>           Below 20 At least 20
#> Automatic 83.33333333 28.57142857
#> Manual    16.66666667 71.42857143

```

The table contains 15 automatic models below the threshold and 4 at or above it, compared with 3 and 10 manual models. `prop.table()` turns counts into proportions. With `margin = 1`, each row supplies its own denominator; multiplying by 100 expresses percentages. Thus $100(4/19) = 21.0526\%$ of automatic models and $100(10/13) = 76.9231\%$ of manual models meet the threshold, a manual-minus-automatic difference of 55.8704 percentage points.

With `margin = 2`, the denominator is instead a column total. Among the 14 threshold-meeting models, $100(10/14) = 71.4286\%$ are manual. “Meeting the threshold among manual models” and “manual among threshold-meeting models” reverse the condition and answer different questions. Naming the denominator keeps the difference clear.

These percentages describe an association among the 32 recorded models. Transmission groups also differ in weight and engine characteristics. Chapter 6 applies proportion and count inference to a declared count model; it cannot by itself identify what changing a transmission would do to fuel economy.

A binary indicator has an additional useful identity: its mean is the proportion coded 1. Thus `mean(cars$mpg >= 20)` is $14/32 = .4375$. Chapter 2’s indicator idea also connects a binary variable to a point-biserial correlation in Section 5.3. Counts and proportions here remain descriptions of the recorded car models.

2.4 Transformations and the Normal model

A linear transformation changes location and units predictably. If $y_i = a + bx_i$, its mean is $a + b\bar{x}$, variance b^2s^2 and SD $|b|s$. The Celsius conversion has $a = -32(5/9)$ and $b = 5/9$.

```

#>      mean_F      SD_F      mean_C      SD_C
#> 77.882352941  9.465269741 25.490196078  5.258483189

```

The mean is 77.8824 degrees Fahrenheit or 25.4902 degrees Celsius; the SDs are 9.4653 and 5.2585. Subtracting a constant affects the mean but leaves the spread unchanged. Multiplying by 5/9 rescales both.

A standardized observation measures distance from the mean in SD units:

$$z_i = \frac{x_i - \bar{x}}{s}, \quad z_{100 \text{ ppb}} = \frac{100 - 42.1293}{32.9879} = 1.7543.$$

`scale()` performs this transformation on a vector. Standardizing changes location and scale, while preserving distributional shape: right-skewed ozone remains right-skewed. This individual z score uses the observations' SD. Section 4's mean-test statistic instead divides by the mean's SE.

R's Normal functions let us compare the observed proportions with a model. `pnorm()` gives a cumulative probability; `lower.tail = FALSE` requests the probability above the supplied value:

```
z_100 <- (100 - mean(ozone)) / sd(ozone)
c(z = z_100, model_above = pnorm(z_100, lower.tail = FALSE),
  observed_above = mean(ozone >= 100),
  model_within_1SD = pnorm(1) - pnorm(-1),
  observed_within_1SD = mean(abs(scale(ozone)) <= 1))
#>           z           model_above      observed_above
#>    1.75430132932      0.03968944406      0.06034482759
#>    model_within_1SD observed_within_1SD
#>    0.68268949214      0.72413793103
```

The upper probability is .0397 under a Normal model with the ozone mean and SD. The observed fraction is 7/116 = .0603. The proportions within one SD are .6827 for a Normal model and .7241 for these data. The model calculation evaluates its supplied assumptions; it does not establish that the distribution describes ozone well. The familiar 68–95–99.7 rule belongs to the Normal model, not to every standardized dataset.

For a separate stipulated score distribution $X \sim N(100, 15^2)$, subtract cumulative areas to obtain an interval probability:

```
c(below_100 = pnorm(100, 100, 15),
  above_140 = pnorm(140, 100, 15, lower.tail = FALSE),
  between_100_120 = pnorm(120, 100, 15) - pnorm(100, 100, 15))
#>      below_100      above_140 between_100_120
#> 0.500000000000 0.003830380568 0.408788780274
qnorm(c(.025, .975), mean = 100, sd = 15)
#> [1] 70.60054023 129.39945977
```

The probabilities are .5000, .0038 and .4088. `qnorm()` reverses the question: it returns a value at a stated cumulative probability. Here the central 95% endpoints are 70.6005 and 129.3995. They bound individual scores under the stipulated model, rather than forming a confidence interval for an unknown mean.

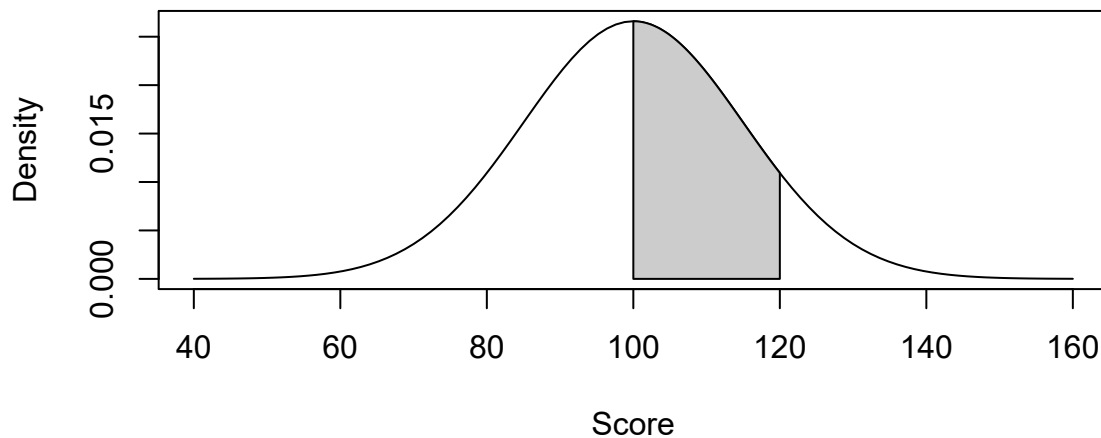


Figure 3: Probability is an area under a stipulated Normal density.

The curve's height is a density. Probability belongs to an area, such as the shaded region, rather than to the height at one exact score.

2.5 A compact distribution catalogue in R

Chapter 2 connected variable forms to probability models. R uses a common naming pattern across these families. A `d` function gives a mass at a discrete value or a continuous density height; `p` gives the cumulative probability $P(X \leq x)$; `q` gives a cumulative quantile; and `r` generates observations from the stated model. The parameters identify which member of a family is being used.

Table 3*Distribution functions respect the model's support and parameters.*

Model	Object and parameters	Base-R family
Bernoulli	One 0/1 response, probability p of 1	<code>binom</code> , with <code>size=1</code>
Binomial	Success count in m independent trials with common p	<code>dbinom</code> , <code>pbinom</code> , <code>qbinom</code> , <code>rbinom</code>
Categorical	One label among categories with stated probabilities	<code>sample()</code> with <code>prob</code> ; a table of masses
Poisson	Nonnegative count with model mean λ	<code>dpois</code> , <code>ppois</code> , <code>qpois</code> , <code>rpois</code>
Normal	Continuous value with mean μ and SD σ	<code>dnorm</code> , <code>pnorm</code> , <code>qnorm</code> , <code>rnorm</code>
Lognormal	Positive value whose log is Normal; parameters on the log scale	<code>dlnorm</code> , <code>plnorm</code> , <code>qlnorm</code> , <code>rlnorm</code>
Logistic	Continuous symmetric value with location μ_L and positive scale s_L	<code>dlogis</code> , <code>plogis</code> , <code>qlogis</code> , <code>rlogis</code>
Student t	Symmetric reference distribution with specified df	<code>dt</code> , <code>pt</code> , <code>qt</code> , <code>rt</code>

For example, $X \sim \text{Binomial}(20, .3)$ counts successes in 20 independent trials with success probability .3. Compare the probability of exactly four with the probability of at most four:

```
c(mass_4 = dbinom(4, size = 20, prob = .3),
  at_most_4 = pbinom(4, size = 20, prob = .3),
  quantile_95 = qbinom(.95, size = 20, prob = .3))
#>      mass_4      at_most_4  quantile_95
#> 0.1304209744 0.2375077789 9.0000000000
set.seed(20517)
rbinom(6, size = 20, prob = .3)
#> [1] 6 8 6 3 5 6
```

The mass is .1304 and the cumulative probability .2375. The .95 quantile is 9, the smallest integer whose cumulative probability reaches .95; discreteness means the attained probability need not be exactly .95. The six draws are 6, 8, 6, 3, 5 and 6. Each draw is a count from one set of 20 trials. The first argument, 6, is the number of counts generated, whereas `size=20` is the number of trials per count. A seed makes these draws reproducible without changing model probabilities.

The same vocabulary works across other supports:

```

c(Bernoulli_1 = dbinom(1, size = 1, prob = .3),
  Poisson_3 = dpois(3, lambda = 2),
  lognormal_below_2 = plnorm(2, meanlog = 0, sdlog = .8),
  logistic_quantile_95 = qlogis(.95, location = 0, scale = 1))
#>      Bernoulli_1      Poisson_3    lognormal_below_2
#>      0.3000000000      0.1804470443      0.8068738906
#> logistic_quantile_95
#>      2.9444389792
labels <- c("A", "B", "C")
probabilities <- c(.2, .3, .5)
set.seed(20517)
sample(labels, size = 6, replace = TRUE, prob = probabilities)
#> [1] "B" "B" "C" "C" "C" "B"

```

The four numerical answers are .3000, .1804, .8069 and 2.9444; the category draws are B, B, C, C, C and B. Categorical probabilities refer to labels. A cumulative probability requires an explicit ordering of those labels; arbitrary category numbers need not have one. Lognormal **meanlog** and **sdlog** describe the logarithm, not the mean and SD of the positive observations. The logistic distribution is continuous and differs from both lognormal and binary outcomes. Its cumulative function later supplies a probability curve in Chapter 11.

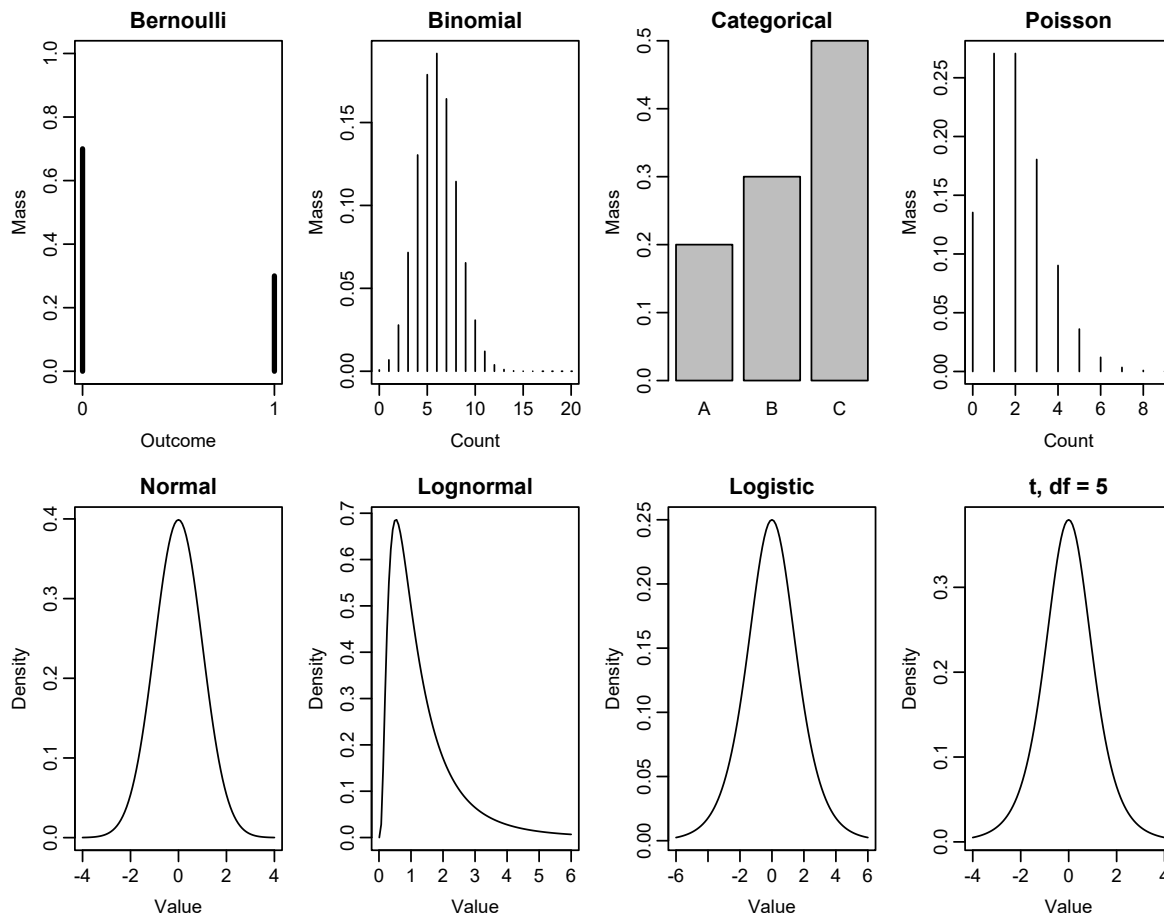


Figure 4: Examples of discrete masses and continuous densities under specified models.

The upper panels assign probability to discrete outcomes, so their masses sum to one over the complete support. The lower panels use area: their densities integrate to one, although any one height need not be less than one. The t curve has heavier tails than the Normal at small df and approaches it as df grows. A family and its parameters together determine these shapes; a function call evaluates that specified model. The Computer Lab provides the figure construction and one-parameter variations as reference examples.

3 Sampling Variation and Standard Errors

3.1 A target, an estimator, and a realized estimate

Chapter 2's invented school system contained 20,000 students. Repeated simple random samples of 100 could give different means while the population mean stayed fixed. Section 1.4 selected IDs without replacement from that finite register. We now change the generating process: instead of selecting IDs from a fixed population, generate independent observations from a probability distribution whose parameters we choose.

For a separate numerical illustration, let the population distribution be Normal with mean 50 and SD 10. Generate 30 independent draws:

```
set.seed(60506)
one_sample <- rnorm(30, mean = 50, sd = 10)
c(model_mean = 50, model_sd = 10,
  sample_mean = mean(one_sample), sample_sd = sd(one_sample))
#>  model_mean      model_sd sample_mean      sample_sd
#> 50.000000000 10.000000000 49.569126166  7.005104058
```

`rnorm()` takes the number of observations, the model mean, and the model SD as arguments. The resulting vector is one sample from that distribution. Its mean is 49.5691 and its SD 7.0051, neither of which has to equal the corresponding model value.

Table 4

One sample of 30 draws from a declared Normal model (seed 60506).

Quantity	Kind	Value
Model mean μ	Parameter	50
Model SD σ	Parameter	10
Sample mean $\bar{x}$	Statistic	49.5691
Sample SD s	Statistic	7.0051

The **parameters** are fixed features of the model; the **statistics** are calculated from the generated observations. Running `rnorm()` again without resetting the seed gives another sample and generally different statistics. Resetting the seed recreates the same draws. A seed controls repeatability of the numerical experiment; it supplies no evidence that the Normal model describes a real study.

A simulation gives us a known target, but a study usually has a target we cannot observe directly. Recall Chapter 3's invented special-practice study. It assigned 120 participants, 60

to Special Practice and 60 to Usual Practice, then measured delayed-assessment scores. The group means were 27.1 and 24.0 points, with SDs 6.0 and 6.4.

The **estimand** is the mean effect of assignment to Special rather than Usual Practice among these 120 participants. Write $Y_i(1)$ for participant i 's potential score under Special assignment and $Y_i(0)$ for the score under Usual assignment. The target and estimator are

$$\Delta\mu = \frac{1}{120} \sum_{i=1}^{120} \{Y_i(1) - Y_i(0)\},$$

$$\widehat{\Delta\mu} = \bar{y}_1 - \bar{y}_0, \quad \text{observed estimate} = 27.1 - 24.0 = 3.1000 \text{ points.}$$

Each participant reveals only the outcome under the assigned condition. We therefore cannot calculate every individual effect and average them. The **estimator**, the difference in assigned-group means, uses the observed outcomes. **Estimation** applies that rule to the data and checks its inputs. The **estimate** is the resulting 3.1000 points.

We can reproduce the arithmetic directly from the supplied summaries:

```
n_special <- 60
n_usual <- 60
mean_special <- 27.1
mean_usual <- 24.0
sd_special <- 6.0
sd_usual <- 6.4
estimate <- mean_special - mean_usual
estimate
#> [1] 3.1
```

The repetition behind uncertainty reassigns the same 120 participants, with exactly 60 in each condition. The participants and their potential outcomes stay fixed; the assigned groups and observed outcomes can vary. This differs from repeatedly sampling new people. Section 4 applies the familiar Welch procedure as an approximate working calibration, retaining Chapter 3's assignment interpretation and conditions.

Table 5
Inferential objects with their R counterparts.

Object	Working definition	In the special-practice study	In R
Estimand	The precisely specified quantity the question seeks	Mean delayed-score difference caused by Special rather than Usual assignment among the 120 participants	Not an object; stated in words before any code
Estimator	The rule applied in each admissible repetition	Special-group mean minus Usual-group mean	The expression <code>mean_special - mean_usual</code>
Estimate	The number the estimator produced	$27.1 - 24.0 = 3.1000$ points	The value of <code>estimate</code>
Statistic	A numerical function of observed data	A group mean, the 3.1000-point estimate	Any computed summary
Parameter	A fixed feature named by the estimand under the chosen interpretation	The participant-specific mean assignment contrast	Unknown here; Section 3.3 changes the target to a known waiting-time variance
Standard error	The modeled spread of the estimator across the declared repetitions	1.1325 points	<code>sqrt(sd_special^2 / n_special + sd_usual^2 / n_usual)</code>

The table distinguishes the scientific target from its computational representation. Storing a result in an object named `estimate` does not define its estimand; that definition comes from the question, participant set, comparison, and outcome.

3.2 The first repeated-sample calculation

Return to Chapter 2’s waiting-time model: independent exponential waits with mean and SD both 10 minutes. The exponential distribution permits many short waits and occasional long ones. R’s `rexp()` uses a **rate**, the reciprocal of the mean, so `rate = 1 / 10` gives a ten-minute mean.

Begin with one sample of five waits and its mean:

```
set.seed(60203)
one_waiting_sample <- rexp(5, rate = 1 / 10)
one_waiting_sample
#> [1] 16.5723079435  3.4747752780  0.1443735324 22.7390197093
```

```
#> [5] 27.9989857599
mean(one_waiting_sample)
#> [1] 14.18589244
```

The first printed vector contains individual waiting times. Its mean compresses the entire sample into one statistic. To see how that statistic varies, repeat the draw-and-average operation:

```
set.seed(60203)
B <- 5000
n_middle <- 30
means_5 <- replicate(B, mean(rexp(5, rate = 1 / 10)))
mean(means_5)
#> [1] 10.01414515
sd(means_5)
#> [1] 4.539459458
```

`replicate()` evaluates its second argument repeatedly and collects the results. Here it returns a vector of 5,000 sample means, not 5,000 individual waits. Resetting the seed before the block lets the first repeated sample reproduce the one-sample example.

The mean of `means_5` is about 10.0141 minutes and its SD about 4.5395 minutes. Next repeat the same operation for sample sizes 30 and 100. Keep the draws in the same order as Chapter 2 to reproduce its numerical results:

```
means_middle <- replicate(
  B, mean(rexp(n_middle, rate = 1 / 10)))
means_100 <- replicate(B, mean(rexp(100, rate = 1 / 10)))
sample_sizes <- c(5, n_middle, 100)
c(mean(means_middle), sd(means_middle))
#> [1] 9.962578922 1.787809834
c(mean(means_100), sd(means_100))
#> [1] 10.002657709 1.007558636
10 / sqrt(sample_sizes)
#> [1] 4.472135955 1.825741858 1.000000000
```

Each histogram below collects 5,000 sample means from one sample size. The display and table use these three vectors; no further random draws are needed. The Computer Lab supplies the plotting construction.

One histogram bar groups sample means in its interval. The vertical density scale divides its count by B times the bin width, so the bar's area is the fraction of repeated means in that interval. In the first panel, each mean comes from five waits; in the last, each comes from 100.

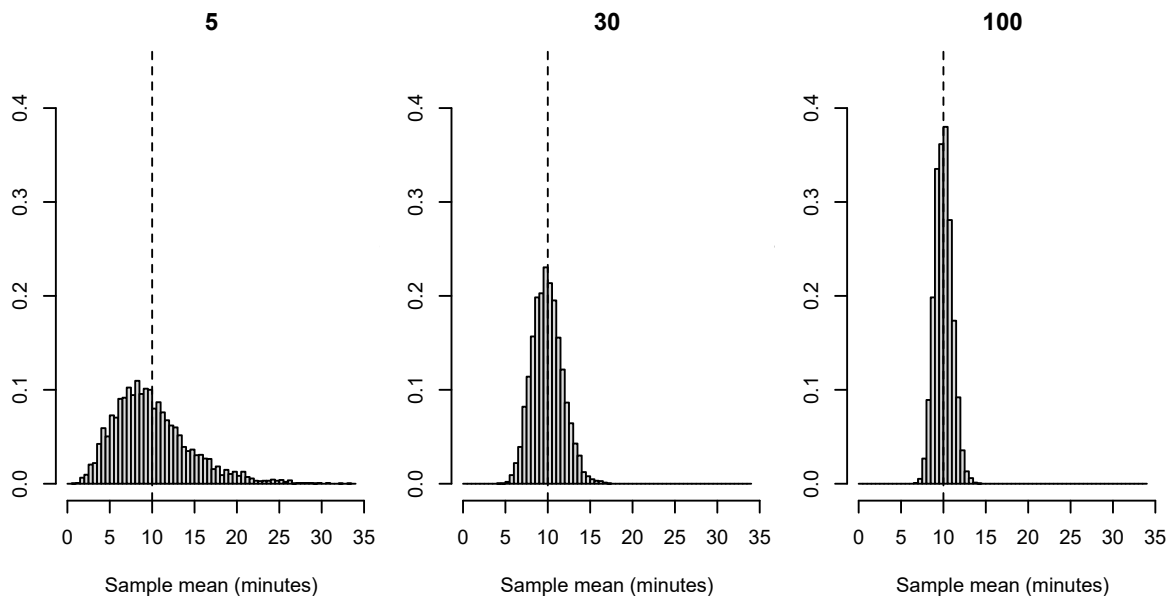


Figure 5: Sampling distributions from 5,000 means at each sample size; the individual waiting-time model is unchanged.

The individual-wait model is unchanged. What changes is the distribution of the mean under each sample size.

Table 6

Centre and spread of the simulated sample means, in minutes.

Sample size	Mean of sample means	Empirical SE	Model SE
5	10.0141	4.5395	4.4721
30	9.9626	1.7878	1.8257
100	10.0027	1.0076	1.0000

The means of sample means are 10.0141, 9.9626, and 10.0027 minutes, close to the fixed model mean. Their empirical SDs are 4.5395, 1.7878, and 1.0076 minutes. The SE label connects these empirical spreads to the model calculation.

A statistic's **standard error** is its SD across the stated repetitions. The SD of the 5,000 generated means approximates that sampling-distribution SD, so it is an *empirical SE*. For independent observations from a common distribution with SD σ , the mean's model SE is

$$SE(\bar{Y}) = \frac{\sigma}{\sqrt{n}}.$$

For these waits,

$$\frac{10}{\sqrt{5}} = 4.4721, \quad \frac{10}{\sqrt{30}} = 1.8257, \quad \frac{10}{\sqrt{100}} = 1.0000 \text{ minutes.}$$

The empirical SEs in Table 6 approximate these values. With finitely many repetitions, they need not agree exactly.

Keep three quantities distinct. The 10-minute SD describes individual waits. The SE describes how sample means vary at a stated sample size. A particular sample's **error**, such as $11.3 - 10 = 1.3000$ minutes for an illustrative mean of 11.3, is one realized departure from the target. An SE summarizes a distribution of possible estimates; it does not tell us the unknown error of a particular estimate.

Increasing n from 5 to 100 multiplies sample size by 20 and divides the SE by $\sqrt{20} \approx 4.4721$. To halve the SE from 1.0000 to 0.5000 minute under this model requires four times as many observations:

$$\frac{10}{\sqrt{400}} = 0.5000 \text{ minute.}$$

Increasing the number of simulation repetitions has a different effect: it shows the same sampling distribution more steadily. It does not change the study sample size or its model SE. To explore the distinction, change `n_middle` or `B` and rerun the simulation from its seed. Change one setting at a time so the cause of the changed display remains clear.

3.3 Bias and precision

An estimator's **expectation** is its probability-weighted average across the declared repetitions. Its **bias** is that expectation minus the target. Chapter 3 illustrated this with an estimator averaging 3.4 when the target was 3.0, a bias of 0.4000. **Precision** concerns how tightly its possible values cluster. An estimator can be centered correctly and still vary widely from one sample to another.

To compare the rules, change the target to the variance of the waiting-time process: 100 squared minutes. For each of 5,000 independent samples of five waits, compute one variance with divisor $n - 1$ and one with divisor n . The Computer Lab reference supplies the complete simulation; its results show the distinction here:

Table 7*Variance estimates from independent exponential samples of five waits.*

Variance rule	Mean over 5,000 samples	Expectation under the model
Divide by $n - 1$	101.2241	100.0000
Divide by n	80.9793	80.0000

Their theoretical expectations are

$$\text{Bias}(\hat{\theta}) = E(\hat{\theta}) - \theta, \quad E(s^2) = \sigma^2,$$

$$E\left\{\frac{1}{n}\sum_{i=1}^n(X_i - \bar{X})^2\right\} = \frac{n-1}{n}\sigma^2 = \frac{4}{5}(100) = 80.0000.$$

The $n - 1$ rule is unbiased under the independent common-distribution model. The n rule has downward bias $\sigma^2/n = 20.0000$ here. The finite-simulation average 101.2241 can differ from 100 even for an unbiased rule: 5,000 samples approximate the expectation rather than enumerating every possible sample.

Both rules can give an estimate far from 100 in a single sample. The n estimate is always four-fifths of the $n - 1$ estimate here, so its spread across repetitions is also smaller, even though its center is wrong. This gives a concrete reason to distinguish precision from bias. An apparently stable estimate can systematically miss the target.

3.4 SE follows the unit and the comparison

An SE follows the observational units that can vary independently. For repeated measurements, alignment comes before subtraction. The sleep documentation describes extra hours of sleep relative to control under two drugs for ten people. ID identifies a person, **group** a drug condition and **extra** the extra hours. Twenty measurement rows represent ten people; these are two drug measurements relative to control, not before-and-after scores. Negative extra sleep means less sleep than under control.

```
all(table(sleep$ID, sleep$group) == 1)
#> [1] TRUE
sleep_wide <- reshape(sleep, idvar = "ID", timevar = "group",
                      direction = "wide")
sleep_wide <- sleep_wide[order(sleep_wide$ID), ]
sleep_d <- sleep_wide$extra.2 - sleep_wide$extra.1
head(sleep_wide, 3)
```

```
#>   ID extra.1 extra.2
#> 1  1     0.7     1.9
#> 2  2    -1.6     0.8
#> 3  3    -0.2     1.1
c(n = length(sleep_d), mean = mean(sleep_d), SD = sd(sleep_d))
#>           n           mean           SD
#> 10.000000000  1.580000000  1.229995483
```

The `all()` check verifies one record per ID and condition. `reshape()` uses the ID to align measurements, creates `extra.1` and `extra.2`, and places each person on one row. Duplicate ID-condition keys need review before reshaping an unfamiliar file. The subtraction is **drug 2 minus drug 1**. For the first person it gives $1.9 - 0.7 = 1.2000$ hours. The ten differences have mean 1.5800 and SD 1.2300 hours. With the same pairs in each calculation, their mean equals the difference of means: $2.3300 - 0.7500 = 1.5800$. Their variance also retains the pairing:

$$s_d^2 = s_2^2 + s_1^2 - 2s_{12}.$$

Here s_{12} is the sample covariance of the two measurements. People with high values in both conditions contribute shared variation that subtraction removes. Verify both identities on the same people:

```
mean(sleep_wide$extra.2) - mean(sleep_wide$extra.1)
#> [1] 1.58
c(variance_difference = var(sleep_d),
  identity = var(sleep_wide$extra.2) + var(sleep_wide$extra.1) -
    2 * cov(sleep_wide$extra.2, sleep_wide$extra.1))
#> variance_difference      identity
#>      1.512888889      1.512888889
```

The variance is 1.5129 squared hours. The covariance is positive, 2.8483, so it reduces the difference variance relative to the sum of the two variances. Separately available means and variances could use different people when a condition is missing; the identities require the same pairs. A high correlation between conditions would describe shared variation, not equality of their means.

For independent observations from the same finite-variance distribution, the sample SD estimates the unknown process SD. The estimated SE is

$$\widehat{SE}(\bar{Y}) = \frac{s}{\sqrt{n}}.$$

For the 116 recorded ozone values, the formula gives $32.9879/\sqrt{116} = 3.0628$ ppb.

```
#> [1] 3.062848185
```

This is an SE **under a working assumption of independent observations from a common process**. The historical daily record does not establish that assumption; nearby days could be related, and the process could change over the period. The calculation demonstrates what the formula returns. Its inferential use needs a justification for the model and target, in addition to the recorded values.

For two independent groups, each mean contributes uncertainty, and their variances add. The special-practice summaries give the same familiar numerical calculation:

$$\begin{aligned}\widehat{SE}(\bar{y}_1 - \bar{y}_0) &= \sqrt{\frac{6.0^2}{60} + \frac{6.4^2}{60}} \\ &= \sqrt{0.6000 + 0.682666\dots} = 1.1325 \text{ points.}\end{aligned}$$

```
standard_error <- sqrt(
  sd_special^2 / n_special + sd_usual^2 / n_usual)
standard_error
#> [1] 1.132548748
```

Its interpretation in this fixed-participant assignment study is different from independent sampling. Exactly 60 of the 120 participants are assigned to each group, so the two assigned-group means are dependent. The exact assignment variance includes a subtraction for variation in individual assignment effects, which cannot be recovered from the observed outcomes. The squared SE above omits that term and estimates an upper bound on assignment variance *on average*. It need not exceed the true variance in every realized study. Chapter 3 develops this distinction and the approximate Welch calibration used in Section 4.

Pairing changes which SD belongs in the numerator. If independent people each supply two measurements, calculate the within-person differences and use their SD:

$$\widehat{SE}(\bar{d}) = \frac{s_d}{\sqrt{n}} = \frac{1.2300}{\sqrt{10}} = 0.3890 \text{ hours.}$$

```
#> [1] 0.3889587239
```

```
#> [1] 0.8490910172
```

The second value, 0.8491 hours, imposes an independence structure that is inappropriate for these paired measurements. It omits their positive within-person covariance. The number of people has stayed at ten; it is the variance calculation, not the count, that changed.

Clustered assignment requires the same attention to the units that can vary independently. Chapter 3's Appendix B, "Paired schools: a comparison of class means," gives the complete

school-blocked feedback example. It has 480 students in 24 classes within 12 schools, with one class per format assigned within each school. It differs from Chapter 1’s assignment of twelve of twenty-four classes across the whole class pool. Here each value is one explanatory-minus-usual class-mean contrast, and the twelve contrasts are $-2, 0, 1, 1, 2, 2, 3, 3, 4, 4, 5$ and 7 :

```
#>           mean           sd           se
#> 2.5000000000 2.3931721057 0.6908492797
```

Their mean is 2.5000 points, SD 2.3932 points, and estimated SE $s_d/\sqrt{12} \approx .6908$ points, using the unrounded SD. This calculation follows the school-block contrasts. Counting 480 student identifiers would not establish 480 independent assignments.

Sampling without replacement also changes uncertainty. If n members are selected by simple random sampling from a finite population of size N , whose variance σ^2 uses denominator N , then

$$SE_{\text{finite}} = \frac{\sigma}{\sqrt{n}} \sqrt{\frac{N-n}{N-1}}.$$

The correction reflects information gained by observing a larger fraction of the fixed population; observing everyone leaves no sampling uncertainty about its mean. Chapter 3’s Appendix A explains the variance conventions and the estimated-SE calculation. With sample variance s^2 using divisor $n-1$, the estimated SE is $(s/\sqrt{n})\sqrt{1-n/N}$. The formula applies specifically to simple random sampling without replacement, rather than to every method that selects observations from a finite population. Across these settings, effective information depends on the estimand, dependence, weighting, and design. Row counts alone cannot determine an uncertainty calculation.

3.5 Concentration and sampling shape

Two results help explain the changing distributions of sample means. The **law of large numbers** concerns concentration: under its conditions, larger samples make the mean more likely to be close to the population mean. The **central limit theorem** concerns shape: under conditions such as independent, identically distributed observations with finite variance, the centered and appropriately scaled sample mean approaches a Normal distribution.

A running mean displays the first idea along one sequence of draws. After draw k , average the first k observations:

The running means are 9.0830 after 10 draws, 7.9852 after 50, 8.8373 after 100, and 9.7207 after 400. This path is irregular: adding observations can temporarily move the mean farther from 10. Concentration is a probabilistic statement as sample size grows, not a promise of monotonic improvement along one sequence or a guarantee at a particular sample size.

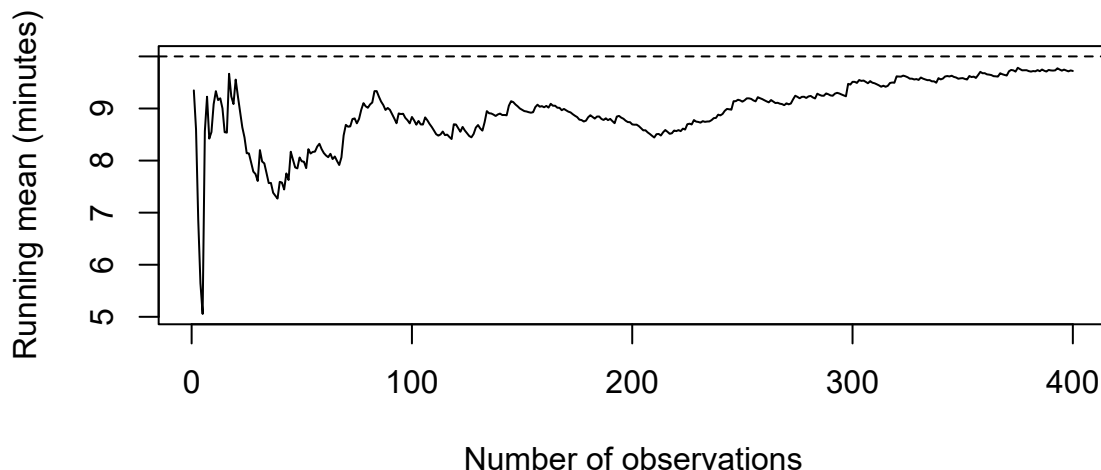


Figure 6: One running mean from independent exponential waits with model mean ten minutes.

The waiting-time histograms in Figure 5 also change shape. Means of larger samples become more nearly symmetric even though individual waits retain their exponential distribution. Concentration describes closeness to the target; the CLT describes the sampling shape after centering and scaling. Neither result makes individual observations Normal. Strong dependence, extreme tails, or a dominating observation can make a familiar approximation poor. There is no universal sample-size cutoff. Chapter 3’s Appendix A gives the formal statements and conditions. The Computer Lab reference applies the reasoning to a second skewed process and supplies the running-mean construction code.

4 Intervals and Tests for Means

4.1 Estimate, SE, critical value, and interval

An estimate answers a numerical question; an interval describes uncertainty about its target under a stated procedure. Before asking R for either, name the outcome and its units, the included observations, the parameter, and the repetition that makes an SE meaningful. The model and design determine the calculation. A function’s default cannot supply them.

The common construction from Chapter 3 is

$$\text{interval} = \hat{\theta} \pm c^* \widehat{\text{SE}}(\hat{\theta}), \quad T = \frac{\hat{\theta} - \theta_0}{\widehat{\text{SE}}(\hat{\theta})}.$$

The estimate and SE have outcome units. Multiplying the SE by the **critical value** c^* gives a margin in those same units. The test statistic instead expresses departure from a null value θ_0 in standard-error units. The confidence level, tail convention and reference distribution determine the critical value and the test's p-value.

For a mean of independent Normal observations whose population SD σ is stipulated independently, the SE is $\sigma/\sqrt{n}$ and the standardized statistic has the standard Normal reference. When the SD is estimated from those observations, use $s/\sqrt{n}$ and a t reference with $n - 1$ degrees of freedom. Estimating the scale introduces uncertainty, reflected in the heavier tails of t (Student, 1908).

The following quantiles leave .025 in the upper tail, as required for a central 95% interval. At low df the t multiplier is noticeably larger; it approaches the Normal multiplier as df increases.

```
c(z_975 = qnorm(.975), t_df5 = qt(.975, 5),
  t_df9 = qt(.975, 9), t_df30 = qt(.975, 30))
#>      z_975      t_df5      t_df9      t_df30
#> 1.959963985 2.570581836 2.262157163 2.042272456
```

These are 1.9600, 2.5706, 2.2622 and 2.0423. A large sample does not make an estimated SD a known population SD; it makes the two reference distributions more similar.

The 32 recorded car models have mean fuel economy 20.0906 mpg. We can use the same observations to compare two explicitly different working assumptions. First, **suppose independently that the process SD is known to be 5 mpg**. This teaching value is not an estimate from `mtcars`. The null process mean is 20 mpg. `abs()` uses the magnitude of the observed z statistic, and `lower.tail = FALSE` requests the upper-tail probability. Doubling it accounts for departures in either direction.

```

mpg <- cars$mpg
n_mpg <- length(mpg)
mean_mpg <- mean(mpg)
sigma_known <- 5
SE_z <- sigma_known / sqrt(n_mpg)
z_mpg <- (mean_mpg - 20) / SE_z
c(estimate = mean_mpg, SE = SE_z, z = z_mpg,
  p = 2 * pnorm(abs(z_mpg), lower.tail = FALSE))
#>      estimate      SE      z      p
#> 20.0906250000 0.8838834765 0.1025304833 0.9183356182
mean_mpg + c(-1, 1) * qnorm(.975) * SE_z
#> [1] 18.35824522 21.82300478

```

The vector `c(-1, 1)` subtracts and adds the same margin, returning the lower and upper endpoint. Now estimate the SD from the records:

```

one_t <- t.test(mpg, mu = 20)
one_t
#>
#> One Sample t-test
#>
#> data:  mpg
#> t = 0.085060036, df = 31, p-value = 0.9327606
#> alternative hypothesis: true mean is not equal to 20
#> 95 percent confidence interval:
#> 17.91767851 22.26357149
#> sample estimates:
#> mean of x
#> 20.090625
c(SD = sd(mpg), SE = one_t$stderr,
  critical = qt(.975, n_mpg - 1))
#>      SD      SE      critical
#> 6.026948052 1.065423959 2.039513446

```

`t.test()` defaults to a two-sided alternative and 95% confidence level. Printing `one_t` gives the method, statistic, df, p-value, stated alternative, interval and estimated mean. The returned object also stores those components: `$estimate`, `$stderr`, `$statistic`, `$parameter`, `$p.value` and `$conf.int`. Here `$parameter` is the reference df, not the estimated process mean.

Table 8*One mean, two explicitly different uncertainty assumptions.*

Procedure	SE	Statistic	df	Two-sided p	95% interval
Known-SD z , $\sigma = 5$	0.8839	0.1025	–	.9183	[18.3582, 21.8230]
Estimated- SD t	1.0654	0.0851	31	.9328	[17.9177, 22.2636]

The t calculation uses observed SD 6.0269 and critical value 2.0395. Both its SE and multiplier exceed those of the stipulated known-SD calculation, so its interval is wider. These calculations illustrate working process models. The historical collection of automobile models does not become a probability sample of a broader population when we call `t.test()`. A broader inference needs an explicitly supported target process and appropriate dependence and distribution conditions.

Both intervals contain 20 mpg, matching their two-sided tests' failure to reject that null at .05. This matching holds when the interval and test use the same method, assumptions and calibration. A p-value is a tail probability under the null and those conditions, not the probability that the null is true. Including 20 establishes neither exact equality nor practical equivalence. An interval for a process mean also differs from a range containing 95% of individual car values.

4.2 Supplied summaries and a planned direction

Chapter 3's separate illustration supplies $n = 25$, mean 52 and sample SD 5 under independent Normal sampling. These summaries are enough to calculate its mean interval and test. We should use the summaries directly rather than inventing observations to pass into `t.test()`. For the null mean 50, the estimated SE is $5/\sqrt{25} = 1$ and the observed t statistic is $(52-50)/1 = 2$.

```
n_sum <- 25
mean_sum <- 52
sd_sum <- 5
SE_sum <- sd_sum / sqrt(n_sum)
t_sum <- (mean_sum - 50) / SE_sum
c(SE = SE_sum, critical = qt(.975, n_sum - 1), t = t_sum,
  two_sided_p = 2 * pt(abs(t_sum), n_sum - 1, lower.tail = FALSE),
  greater_p = pt(t_sum, n_sum - 1, lower.tail = FALSE))
#>      SE      critical      t      two_sided_p
#> 1.00000000000 2.06389856163 2.00000000000 0.05693984994
#>      greater_p
```

```
#> 0.02846992497
mean_sum + c(-1, 1) * qt(.975, n_sum - 1) * SE_sum
#> [1] 49.93610144 54.06389856
```

The SE is 1.0000, critical t is 2.0639, and the interval is [49.9361, 54.0639]. The two-sided p-value is .0569; the upper-tail p-value for the alternative $\mu > 50$ is .0285. These differ because they answer different prespecified questions. A directional alternative must be chosen from the question before inspecting the result. For a negative statistic, the upper-tail probability would exceed .5; blindly halving a two-sided p-value would then give the wrong tail.

The two-sided interval contains 50, consistent with the two-sided test at .05. The upper-tail test does not match that same two-sided 95% interval. If a task stipulates the rounded critical value 2.064, its endpoints are [49.9360, 54.0640]. Use the supplied value for that task and retain the software quantile when reproducing the software procedure; this is a rounding difference, not a changed conclusion.

4.3 Paired observations and complete pairs

The sleep measurements return us to a comparison whose unit is a person. Section 3 aligned measurements by ID into `sleep_wide` and defined `sleep_d` as drug 2 minus drug 1. There are ten people with complete pairs, not twenty independent people. Their condition means are 0.7500 and 2.3300 hours, and the mean of their aligned differences is 1.5800 hours. Nine differences are positive and one is zero; they range from 0 to 4.6 hours.

The inferential target is a process mean difference μ_D . Under a working model of independent Normal differences between people, $s_d/\sqrt{n}$ estimates the SE of their mean and a t reference has $n - 1$ df. Normality concerns the differences. Dependence between the two measurements on one person is preserved in their difference.

The observed difference SD is 1.2300 hours and its mean SE is 0.3890 hours. Thus the interval and zero-null statistic are

$$\bar{d} \pm t_{.975,9} \frac{s_d}{\sqrt{10}} = [0.7001, 2.4599] \text{ hours}, \quad t = \frac{\bar{d}}{s_d/\sqrt{10}} = 4.0621.$$

`paired = TRUE` tells R to subtract measurements at corresponding positions. The ID-based alignment performed earlier is what makes those positions correct. `complete.cases()` checks that both required measurements are present on the same row. Here every pair is complete.

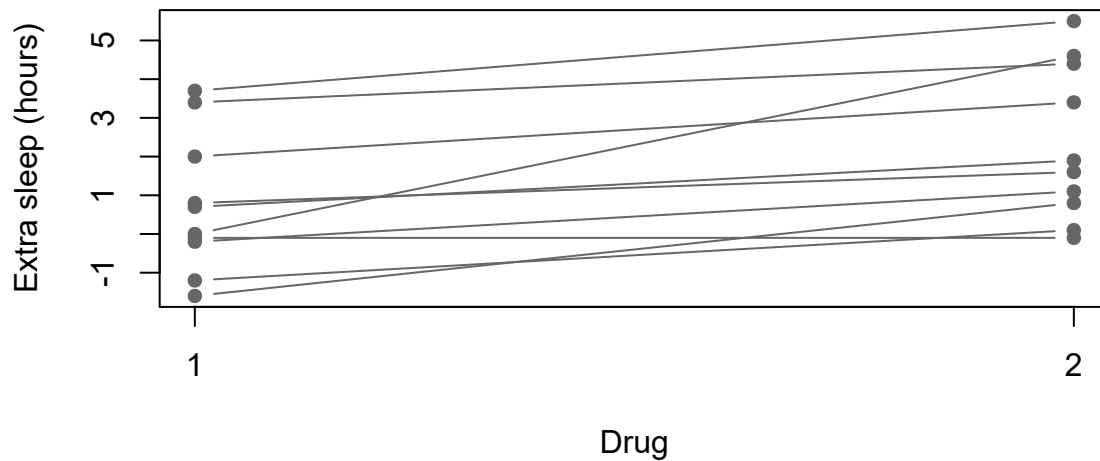


Figure 7: Two measurements for each of ten people.

```
complete_sleep <- complete.cases(sleep_wide[c("extra.1", "extra.2")])
table(complete_sleep)
#> complete_sleep
#> TRUE
#> 10
paired_t <- t.test(sleep_wide$extra.2[complete_sleep],
  sleep_wide$extra.1[complete_sleep], paired = TRUE)
c(t = unname(paired_t$statistic), df = unname(paired_t$parameter),
  p = paired_t$p.value)
#>          t          df          p
#> 4.062127683382 9.000000000000 0.002832890197
paired_t$conf.int
#> [1] 0.7001142367 2.4598857633
#> attr("conf.level")
#> [1] 0.95
c(estimate = mean(sleep_d), SE = sd(sleep_d) / sqrt(length(sleep_d)),
  critical = qt(.975, length(sleep_d) - 1))
#>      estimate      SE      critical
#> 1.5800000000 0.3889587239 2.2621571628
t.test(sleep_d, conf.level = .99)$conf.int
#> [1] 0.3159481149 2.8440518851
```

```
#> attr("conf.level")
#> [1] 0.99
```

The extracted components give $t(9) = 4.0621$, $p = .0028$ and the 95% interval $[0.7001, 2.4599]$ hours. The critical t is 2.2622. A paired t calculation is a one-sample t calculation on the within-person differences, so the last call uses `sleep_d` directly. Increasing confidence to 99% widens its interval to $[0.3159, 2.8441]$; the observations and estimate stay unchanged. The Computer Lab checks the two interfaces on the same pairs.

With missing measurements, retain the rows having both values before forming differences. Removing missing entries independently from each column could align different people. Complete-pair analysis also changes who contributes and may need a missingness justification; a software exclusion is not that justification.

The model interval concerns a process mean, not a guarantee of benefit for every person. Ten differences give limited protection against non-Normal tails or influential cases. Neither the paired call nor the dataset establishes random sampling, treatment order, absence of carryover, or other conditions needed for a causal interpretation.

The same arithmetic reproduces Chapter 3's invented eight-student illustration. Keep after minus before so that improvement is positive:

```
before <- c(50, 54, 48, 60, 55, 58, 62, 53)
after  <- c(49, 54, 49, 62, 57, 61, 66, 58)
change <- after - before
student_t <- t.test(after, before, paired = TRUE)
c(t = unname(student_t$statistic), df = unname(student_t$parameter),
  p = student_t$p.value)
#>               t              df              p
#> 2.82842712475 7.00000000000 0.02546356168
student_t$conf.int
#> [1] 0.3279581567 3.6720418433
#> attr("conf.level")
#> [1] 0.95
c(mean = mean(change), SD = sd(change),
  SE = sd(change) / sqrt(length(change)))
#>      mean      SD      SE
#> 2.0000000000 2.0000000000 0.7071067812
```

The mean change is 2.0000 points, SD 2.0000 and SE 0.7071; the test gives $t(7) = 2.8284$, $p = .0255$, and interval $[0.3280, 3.6720]$. Pairing preserves the comparison; design and timing evidence still determine whether the change can be attributed to an intervention. Chapter 3's paired-school appendix applies the same form to school-specific class-mean contrasts. Its

twelve school blocks, assignment interpretation and complete data remain there; the Computer Lab reference reproduces that calculation without treating 480 student scores as independent assignment contrasts.

4.4 Two independent groups and Welch's method

Now compare separate automatic and manual car models. Let $\Delta\mu = \mu_A - \mu_M$ be the automatic-minus-manual process mean mpg difference. Under an independent-groups working model, the two means contribute separate variance terms. Welch's method estimates both and allows their group variances to differ (Welch, 1947).

$$\widehat{\Delta\mu} = \bar{y}_A - \bar{y}_M, \quad a = \frac{s_A^2}{n_A}, \quad b = \frac{s_M^2}{n_M}, \quad \widehat{SE} = \sqrt{a + b},$$

$$\nu = \frac{(a + b)^2}{a^2/(n_A - 1) + b^2/(n_M - 1)}.$$

The Welch–Satterthwaite df ν account approximately for the two estimated variance terms. They need not be an integer. R codes automatic transmission as `am == 0` and manual as `am == 1`; extracting the two vectors in a stated order makes the subtraction unambiguous.

```
auto <- cars$mpg[cars$am == 0]
manual <- cars$mpg[cars$am == 1]
a <- var(auto) / length(auto)
b <- var(manual) / length(manual)
df_welch <- (a + b)^2 /
  (a^2 / (length(auto) - 1) + b^2 / (length(manual) - 1))
c(n_auto = length(auto), n_manual = length(manual),
  mean_auto = mean(auto), mean_manual = mean(manual),
  difference = mean(auto) - mean(manual), SE = sqrt(a + b),
  df = df_welch)
#>      n_auto      n_manual  mean_auto mean_manual difference
#> 19.000000000 13.000000000 17.147368421 24.392307692 -7.244939271
#>      SE      df
#> 1.923202134 18.332251638
welch <- t.test(auto, manual)
c(t = unname(welch$statistic), df = unname(welch$parameter),
  p = welch$p.value)
#>      t      df      p
#> -3.767123145145 18.332251638400 0.001373638333
welch$conf.int
#> [1] -11.280194355 -3.209684187
```

```
#> attr("conf.level")
#> [1] 0.95
```

There are 19 automatic and 13 manual models, with means 17.1474 and 24.3923 mpg. The difference is -7.2449 mpg, SE 1.9232 and approximate df 18.3323. The two-sided p-value is .0014 and the 95% interval is $[-11.2802, -3.2097]$. The negative sign describes the declared order. Reversing it would reverse the estimate and negate and exchange the endpoints, while preserving the two-sided p-value.

The two-sample `t.test()` default is Welch's method. Pooled-variance inference, requested with `var.equal = TRUE`, gives SE 1.7644, df 30, $p=.0003$ and interval $[-10.8484, -3.6415]$.

```
#>           SE           df.df           p
#> 1.764421632e+00 3.000000000e+01 2.850207439e-04
#> [1] -10.848368971 -3.641509572
#> attr("conf.level")
#> [1] 0.95
```

The pooled t reference is exact for independent Normal groups with a common variance. Welch's t reference remains approximate even for independent unequal-variance Normal groups. The estimate and observations are unchanged; the variance assumptions and calibration differ. A smaller p-value is not a reason to select the common-variance assumption. Neither method resolves differences in weight, engine and other features associated with transmission. These observational comparisons do not isolate a transmission effect.

We can also complete Chapter 3's special-practice calculation from its supplied summaries. The invented study assigned the same 120 participants, 60 to each format, and observed every delayed assessment. Its estimand is the mean effect of Special rather than Usual assignment among those participants. The repetition reassigns the fixed people; it does not independently sample two new groups.

Section 3 explained why the familiar squared SE estimates an upper bound on assignment variance on average. We retain Chapter 3's approximate Welch working reference for that assignment analysis. The shared arithmetic does not replace its design-based interpretation. The earlier objects hold means 27.1 and 24.0 and SDs 6.0 and 6.4:

```

a_special <- sd_special^2 / n_special
a_usual <- sd_usual^2 / n_usual
df_special <- (a_special + a_usual)^2 /
  (a_special^2 / (n_special - 1) + a_usual^2 / (n_usual - 1))
t_special <- estimate / standard_error
c(estimate = estimate, SE = standard_error, df = df_special,
  critical = qt(.975, df_special), t = t_special,
  p = 2 * pt(abs(t_special), df_special, lower.tail = FALSE))
#>      estimate      SE      df      critical
#> 3.100000000e+00 1.132548748e+00 1.175118930e+02 1.980357462e+00
#>      t      p
#> 2.737189022e+00 7.160041576e-03
estimate + c(-1, 1) * qt(.975, df_special) * standard_error
#> [1] 0.8571486358 5.3428513642

```

The estimate is 3.1000 points, SE 1.1325, df 117.5119 and critical t 1.9804. The statistic is 2.7372, $p=.0072$, with 95% interval [0.8571, 5.3429]. This interval includes effects below a stated two-point importance benchmark as well as above it. Evidence against a zero effect therefore supports less than a claim that the mean improvement exceeds two points. Generalization beyond the participating students needs further support. The supplied rounded summaries are also distinct from the companion CSV's slightly different SDs; they should not silently be substituted for one another.

4.5 Coverage belongs to the procedure

What makes an interval procedure a 95% procedure? We can inspect the answer by generating data from a known process. Let independent differences be Normal with mean 1.5 hours and SD 1.2 hours. These are teaching constants, not a reconstruction of the historical sleep study. Draw ten differences, estimate their mean and SE, and form a t interval.

```

#> [1] 1.021299046 2.211947792
#> [1] TRUE

```

This first interval is [1.0213, 2.2119] hours and contains 1.5, so the logical check returns **TRUE**: both endpoint conditions hold. A repeated experiment generates fresh observations and repeats these steps, keeping the true mean fixed. Recalculating the interval on the same observations would produce the same interval, not a coverage experiment. The Computer Lab reference supplies the full loop and changes in sample size or repetitions.

Table 9

Coverage of mean intervals from the Normal process, using 2,000 repetitions at each sample size and resetting seed 60610 for each row.

n	Mean of estimates	Empirical SE	Model SE	Coverage
10	1.5015	0.3766	0.3795	.9515
20	1.5006	0.2591	0.2683	.9510
40	1.5014	0.1886	0.1897	.9460

The empirical SE is the SD of the generated means; the model SE is $1.2/\sqrt{n}$. Coverage is the fraction of generated intervals containing 1.5. Under the declared Normal model, the t procedure has exact 95% coverage. A finite simulation approximates that value, so its observed fraction need not equal .9500.

The fraction has its own **Monte Carlo SE**. Treat each independent generated study's coverage indicator as a Bernoulli outcome. At coverage probability .95 and $B = 2000$ repetitions,

$$\text{MCSE}(\hat{p}_{\text{cover}}) = \sqrt{\frac{.95(.05)}{2000}} = 0.0049.$$

Increasing B reduces simulation noise; quadrupling it halves this MCSE. Increasing n supplies more information in each study and narrows intervals: changing 10 to 40 halves the model SE. It does not systematically raise the nominal 95% coverage above 95%. The individual-difference SD, the mean's SE and the simulation's MCSE describe different sources of variation.

Chapter 3's coverage display shows the same fixed-parameter reasoning. For one observed interval the parameter is either inside or outside; the 95% describes repeated use of the procedure. A seed makes the simulation repeatable, and suitable model and design information make an interval interpretation defensible in an actual study.

5 Correlations and Their Inference

5.1 Begin with the plotted observations

Mean comparisons group observations by a category. A relationship between two quantitative variables instead begins with pairs of values. For the 32 car models, `wt` measures weight in thousands of pounds and `mpg` fuel economy in miles per US gallon. Keep the same complete rows for the graph and every numerical description of that pair:

```
car_pairs <- mtcars[complete.cases(mtcars[c("wt", "mpg")]), ]
c(n = nrow(car_pairs), covariance = cov(car_pairs$wt, car_pairs$mpg),
  correlation = cor(car_pairs$wt, car_pairs$mpg))
#>           n      covariance      correlation
#> 32.0000000000 -5.1166846774 -0.8676593765
```

```
plot(car_pairs$wt, car_pairs$mpg, pch = 16,
      xlab = "Weight (thousand pounds)", ylab = "Miles per gallon")
```

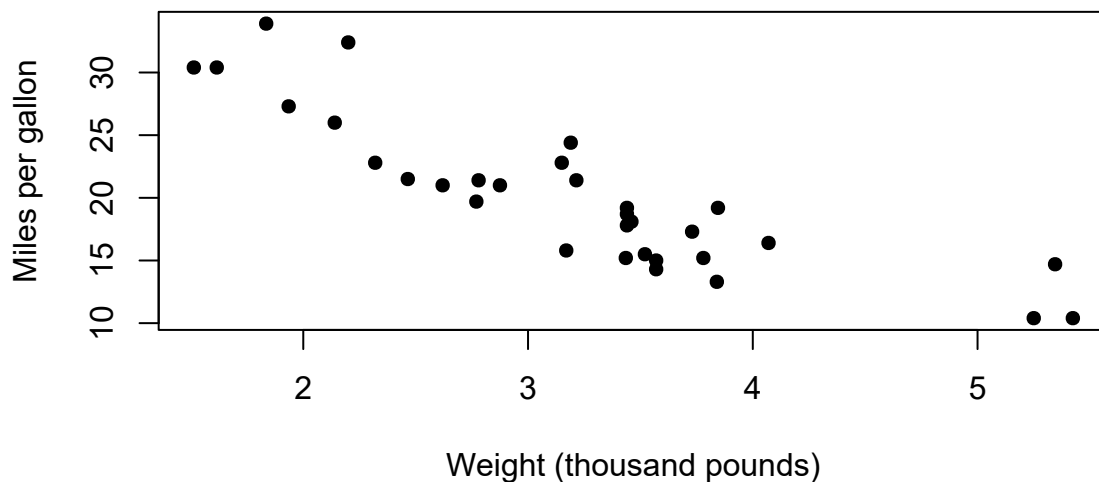


Figure 8: Mileage and weight for 32 recorded car models.

Read direction, strength and form before reducing the display to a coefficient. Here heavier models generally have lower mpg. Inspect unusual observations, the range of weights, possible subgroups and changes in spread. A coefficient compresses these features, so the plot helps determine what that number can usefully describe.

Covariance averages products of paired deviations. On the same n complete observations, sample covariance and Pearson correlation are

$$s_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{n - 1}, \quad r_{xy} = \frac{s_{xy}}{s_x s_y}.$$

A pair contributes positively when both values are above their means or both are below; opposite deviations contribute negatively. Covariance retains the product of the two units. Dividing by the SDs gives a unitless coefficient between -1 and 1 when both variables have nonzero spread. For $x = (1, 2, 3)$ and $y = (2, 4, 6)$, the deviation products sum to four, covariance is $4/(3 - 1) = 2.0000$, and the SDs are one and two. Thus $r = 1.0000$: all points lie on one increasing line.

The car covariance is -5.1167 thousand-pounds times mpg and $r = -.8677$. Changing the weight unit changes covariance predictably.

```
#> covariance_pounds correlation_pounds
#> -5116.6846774194 -0.8676593765
```

Covariance multiplies by 1,000; the weight SD does too, leaving the correlation unchanged. Adding a constant changes neither covariance nor correlation. Multiplying one variable by a negative constant reverses both signs.

A small correlation need not mean little relationship. The five points $x = (-2, -1, 0, 1, 2)$ and $y = x^2 = (4, 1, 0, 1, 4)$ follow an exact curve while having zero covariance and correlation.

```
#> covariance      r
#>          0      0
```

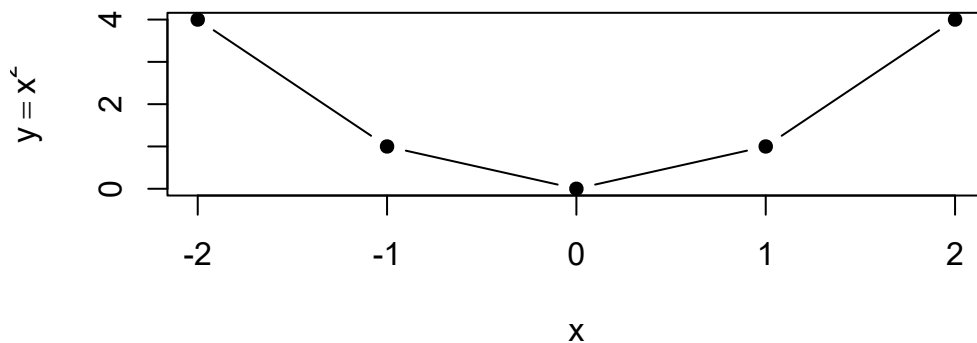


Figure 9: An exact curved relationship with zero Pearson correlation.

Positive and negative cross-products cancel by symmetry, although knowing x determines y . Pearson correlation describes linear association, so “the variables are unrelated” would overstate this result.

Chapter 2’s gallery gives a compact visual reference. Each panel below contains 80 simulated pairs and uses the same axes. The generating correlations are $-.9, -.5, 0, .3, .6, .9$, but each label reports the correlation of the displayed observations. Sampling variation separates a model parameter from the realized sample coefficient.

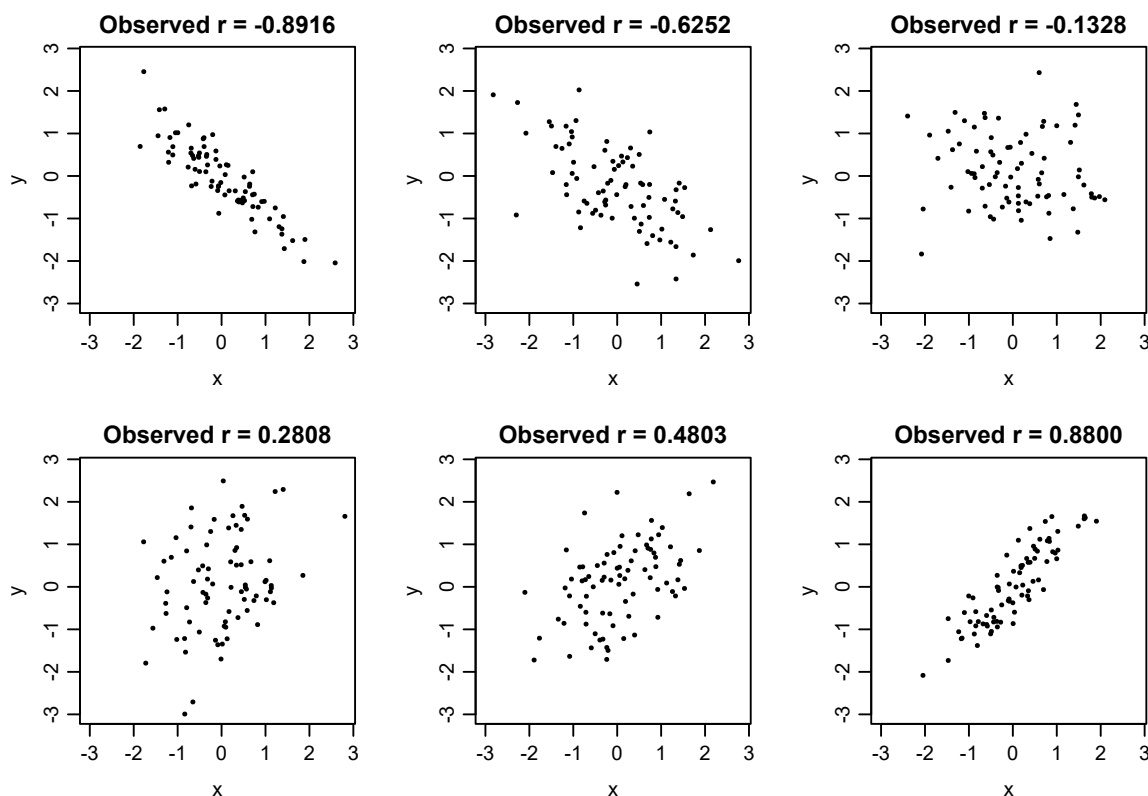


Figure 10: Realized samples with different directions and strengths of linear association.

The sign gives direction and the absolute value summarizes linear strength. The model with generating correlation zero happened to produce a slightly negative sample coefficient. A generated display illustrates the model; it provides no evidence that actual study observations follow that model. The lab reference supplies the complete generating and plotting code.

5.2 Covariance and correlation matrices

A covariance matrix extends the pairwise calculation to several variables. Its diagonal contains variances, and its off-diagonal entries contain covariances. A correlation matrix standardizes each entry; diagonal entries are one for variables with positive variance.

Select one common complete-case set before forming either matrix:

```
variables <- c("mpg", "hp", "wt")
X <- mtcars[complete.cases(mtcars[variables]), variables]
S <- cov(X)
R <- cov2cor(S)
s <- sqrt(diag(S))
S_rebuilt <- R * outer(s, s)
c(n = nrow(X), largest_reconstruction_error =
  max(abs(S_rebuilt - S)))
#>                                n largest_reconstruction_error
#>                3.200000000e+01                9.094947018e-13
```

All 32 rows are complete for fuel economy, gross horsepower (**hp**) and weight. `diag(S)` extracts the variances; taking their square roots gives SDs. `outer(s,s)` forms every pairwise SD product. Multiplying it entry by entry by `R` restores the covariance matrix:

Table 10

Sample covariance matrix for the same 32 car models.

Variable	mpg	hp	wt
mpg	36.3241	−320.7321	−5.1167
hp	−320.7321	4700.8669	44.1927
wt	−5.1167	44.1927	0.9574

Table 11

Corresponding Pearson correlation matrix.

Variable	mpg	hp	wt
mpg	1.0000	−.7762	−.8677
hp	−.7762	1.0000	.6587
wt	−.8677	.6587	1.0000

For example, the mpg variance has squared-mpg units, while its covariance with horsepower has mpg-times-horsepower units. Correlations remove those units. If D is the diagonal matrix of positive sample SDs, the two operations are

$$R = D^{-1}SD^{-1}, \quad S = DRD.$$

The reconstructed values agree up to floating-point precision. Correlations alone cannot recover the original covariance matrix: the SDs and their units are also needed. A constant column has zero SD and an undefined correlation; detect it before dividing or converting.

Using common rows matters when measurements are missing. Computing each entry from a different pairwise subset can produce a matrix that does not describe one joint sample and is not positive semidefinite, so it can imply an impossible negative variance for a linear combination. For airquality's ozone, solar radiation and wind, the individual recorded counts are 116, 146 and 153, but the common set contains 111 days. The Computer Lab reference computes that common-case matrix and checks that standardizing $\mathbf{X}$ leaves its correlation matrix unchanged.

5.3 Pearson, rank, and point-biserial coefficients

Pearson correlation uses the measured scales and describes linear association. **Spearman correlation** applies correlation to ranks and describes monotone association: values tend to move consistently in one direction, even when the pattern is curved. **Kendall correlation** compares concordant and discordant ordering of pairs, with an adjustment for ties. The coefficients address related but distinct features.

```
#>      Pearson      Spearman      Kendall
#> -0.7761683718 -0.8946646457 -0.7428125061
#> point_biserial reversed_coding
#>    0.5998324295   -0.5998324295
```

For mpg and horsepower, the coefficients are $-.7762$, $-.8947$, $-.7428$. Tied horsepower values retain their tied ranks; their presence also affects test calibration. The different magnitudes do not make the largest coefficient the universally best description. The question and plot determine which association is relevant.

A **point-biserial correlation** is Pearson correlation with one binary variable coded 0/1. Here **am=1** means manual and **am=0** automatic; $r = .5998$ corresponds to means 24.3923 and 17.1474 mpg, respectively. Reversing the coding reverses the coefficient's sign while leaving the same two groups. For more than two categories, arbitrary numerical labels generally do not define a meaningful quantitative scale for correlation. Coding must follow the variable's role.

5.4 A selected correlation test

The descriptive weight–mpg correlation concerns the recorded models. Inference adds a target process correlation ρ and a supporting model. Suppose we selected the two-sided null $H_0 : \rho = 0$ before examining the result, with level $\alpha = .05$. Under independent bivariate-Normal sampling, the zero-correlation statistic has an exact t_{n-2} reference:

$$t = r \sqrt{\frac{n-2}{1-r^2}}, \quad df = n-2.$$

Use the same 32 complete pairs as in the graph:

```
pearson_test <- cor.test(car_pairs$wt, car_pairs$mpg,
                          method = "pearson")
c(r = unname(pearson_test$estimate), t = unname(pearson_test$statistic),
  df = unname(pearson_test$parameter), p = pearson_test$p.value)
#>           r           t           df
#> -8.676593765e-01 -9.559044147e+00  3.000000000e+01
#>           p
#> 1.293958701e-10
pearson_test$conf.int
#> [1] -0.9338264133 -0.7440871965
#> attr(,"conf.level")
#> [1] 0.95
r_wt <- cor(car_pairs$wt, car_pairs$mpg)
t_wt <- r_wt * sqrt((nrow(car_pairs) - 2) / (1 - r_wt^2))
c(t = t_wt, df = nrow(car_pairs) - 2,
  p = 2 * pt(abs(t_wt), nrow(car_pairs) - 2, lower.tail = FALSE))
#>           t           df           p
#> -9.559044147e+00  3.000000000e+01  1.293958701e-10
```

The coefficient is $r = -.8677$, with $t(30) = -9.5590$ and $p < .0001$. R prints the very small p-value in scientific notation. The result supplies evidence against zero process correlation under the stated reference model; a small p-value does not establish that model or turn the historical car records into a probability sample. Normality is unnecessary to calculate descriptive r . Independence, shape, influential cases and the observed range matter for inference.

The printed 95% interval is $[-.9338, -.7441]$. It uses Fisher's transformation, a different approximation from the exact zero-null test. Transform the bounded correlation, form an approximately symmetric interval on that scale, then transform back:

$$CI_\rho = \tanh \left\{ \operatorname{atanh}(r) \pm \frac{z_{.975}}{\sqrt{n-3}} \right\}.$$

```
fisher_interval <- tanh(atanh(r_wt) +
  c(-1, 1) * qnorm(.975) / sqrt(nrow(car_pairs) - 3))
fisher_interval
#> [1] -0.9338264133 -0.7440871965
```

This interval requires at least four complete pairs and remains approximate even under bivariate Normality. Substantial nonlinearity can make linear correlation a poor answer to the substantive question, regardless of the numerical width.

For a preselected rank-association question, keep the coefficient beside the test and retain the statistic's native name. The mpg and horsepower data contain ties, so the approximate calibrations are requested explicitly with `exact = FALSE`.

```
#>           S           p
#> 1.033729031e+04 5.085969431e-12
#>           z           p
#> -5.871005272e+00 4.331604949e-09
```

Spearman's output reports $S = 10337.2903$, while Kendall's reports $z = -5.8710$; both give $p < .0001$. These are not Pearson t statistics. The rank tests still require an appropriate independent-observation model and a question about the relevant association. They do not supply the same Fisher interval as the Pearson call. Choosing a method because its p -value is smallest changes the testing procedure; a matrix of unplanned tests also creates a comparison family, developed in Chapter 6.

For a binary predictor, the point-biserial t statistic equals a pooled two-group mean-test statistic. Keep manual first, matching the positive indicator coding:

```
#> point_biserial_t      pooled_t      df
#> 4.106126983e+00 4.106126983e+00 3.000000000e+01
#>           p
#> 2.850207439e-04
```

Both give $t(30) = 4.1061$ and two-sided $p = .0003$. The supporting model is independent, conditionally Normal outcomes with equal variance in the two groups; a binary predictor itself cannot have a bivariate Normal distribution with the outcome. This equivalence is to the pooled procedure and generally does not hold for Welch's test. Chapter 7 connects it further to a regression slope.

5.5 One-control partial correlation

Chapter 2 distinguished changing the cases from adjusting a relationship on the same cases. For variables X, Y, Z , the **partial correlation** of X and Y controlling linearly for Z is

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ}r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}.$$

Its supplied practice-time, score and baseline correlations were $r_{XY} = .60, r_{XZ} = .50, r_{YZ} = .60$. Compute that descriptive adjustment first, then apply the same formula to mpg and horsepower controlling for weight:

```
r_xy <- .60; r_xz <- .50; r_yz <- .60
(r_xy - r_xz * r_yz) / sqrt((1 - r_xz^2) * (1 - r_yz^2))
#> [1] 0.4330127019
r_partial <- (R["mpg", "hp"] - R["mpg", "wt"] * R["hp", "wt"]) /
  sqrt((1 - R["mpg", "wt"]^2) * (1 - R["hp", "wt"]^2))
r_partial
#> [1] -0.5469926242
```

The supplied correlations give .4330. For the car models, the mpg–hp coefficient changes from $-.7762$ to $-.5470$ after linear adjustment for weight. The adjustment uses the same 32 rows; it does not exclude heavy cars or replace their observations.

There is a second way to see the calculation. Fit a straight line from weight to each variable and retain its **residuals**, the observed values minus their fitted values. `lm()` fits the line specified by its formula; `mpg ~ wt` places the outcome left of `~` and the predictor right of it. Each fit below includes an intercept. `residuals()` returns what remains after that fitted linear relationship:

```
resid_mpg <- residuals(lm(mpg ~ wt, data = X))
resid_hp <- residuals(lm(hp ~ wt, data = X))
c(formula = r_partial, residual_correlation = cor(resid_mpg, resid_hp))
#>          formula residual_correlation
#>      -0.5469926242      -0.5469926242
```

Both calculations give $-.5470$. A model with several predictors develops this interpretation further in Chapter 8. Here the important distinction is that specified linear relationships have been removed. This operation does not remove all confounding, omitted nonlinear relationships or measurement problems, and it does not establish a causal effect of horsepower.

If the partial coefficient is the preselected inferential question, the reference must account for fitting the control. With k controls and n common observations, the classical statistic uses $n - k - 2$ df:

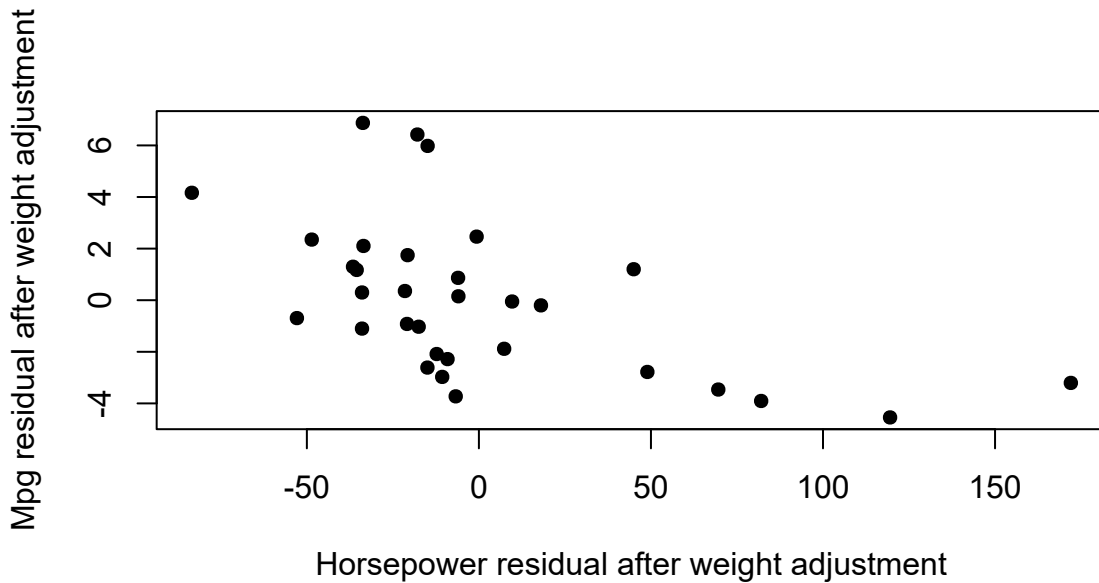


Figure 11: Mileage and horsepower after separately fitting their linear relationships with weight.

$$t = r_{XY \cdot Z} \sqrt{\frac{n - k - 2}{1 - r_{XY \cdot Z}^2}}.$$

```
k_controls <- 1
df_partial <- nrow(X) - k_controls - 2
t_partial <- r_partial * sqrt(df_partial / (1 - r_partial^2))
p_partial <- 2 * pt(abs(t_partial), df_partial, lower.tail = FALSE)
c(r = r_partial, t = t_partial, df = df_partial, p = p_partial)
#>      r      t      df      p
#> -0.546992624170 -3.518711910209 29.000000000000 0.001451228532
```

The result is $t = -3.5187$, $df = 29$ and $p = .0015$. This exact classical calibration requires independent observations, full-rank linear adjustment (without redundant fitted terms) and the relevant Normal conditional-error model, or a multivariate-Normal model. Applying ordinary `cor.test()` to the fitted residual vectors would use $n - 2 = 30$ df and ignore the estimated control. The correctly adjusted reference makes the fitting step visible.

A sensitivity analysis asks a different question. Restricting the car record to weight at most 5,000 pounds changes its cases and gives $r = -.8792$ between weight and mpg. Removing three ozone days above 120 ppb changes its mean to 39.4867, SD to 28.8942 and median to

30.0000. These are consequences of stated restrictions, not reasons by themselves to delete observations. The lab reference compares them with the full recorded sets. Linear adjustment and case selection remain distinct, even when both change a reported association.

6 Report the Answer

6.1 A result connects numbers to a question

A statistical report returns from the function output to the question. Name the people or records, outcome and units, comparison, estimate and uncertainty. Then state the conditions and scope that support the interpretation. The sleep result can be reported as follows:

For the ten recorded people, extra sleep under drug 2 averaged 1.5800 hours more than under drug 1. The SD of the ten within-person differences was 1.2300 hours. Under an independent Normal-differences working model, the mean difference has SE 0.3890 hours and a 95% t interval of [0.7001, 2.4599] hours ($t(9) = 4.0621$, two-sided $p = .0028$). This uncertainty concerns the stated process mean. A causal account also requires information about treatment assignment, order and carryover.

The first two sentences describe the recorded people. The next adds model-based uncertainty for a named target, rather than a range of individual responses. The final sentence identifies specific information needed for a causal claim. A successful computation supplies the result; its study setting supplies the argument for using it (Peng, 2011).

6.2 Connect a report to its source

A saved script records calculations. A Quarto source places them beside the prose that explains them, so a render executes the code and inserts its output. The prepared two-file example uses a short `.qmd` source and an accompanying bibliography. Its paired analysis uses these same data, subtraction direction, model and base-R commands. One chunk computes the result; an inline value inserts a number into a sentence; a table or graph helps readers see what it describes.

During the final Computer Lab round, render the prepared source, change confidence from .95 to .99 and revise the corresponding sentence. The interval becomes [0.3159, 2.8441] while the estimate stays 1.5800 hours. That small change reveals the connection between an input, a calculation, output and interpretation. The lab reference explains the header, chunks, cross-references, citations and report-support installation. Its first HTML render requires no PDF typesetting setup.

When a data-selection rule or a method changes, computed values can update automatically; the interpretation still needs to be reread. A reproducible report keeps question, data, code,

output and explanation consistent. Rendering demonstrates that a document can be produced, while the evidence and reasoning determine what its claims establish.

7 Reading Bridge

When a study reports a mean, interval or association, identify what was recorded, which observations entered the calculation, the statistic or comparison and its units. Next locate the repetition or model that supports the SE, reference distribution, interval or test. Finally separate the numerical result from the assumptions needed for population, practical or causal claims. R reproduces a computation; the study's evidence and design support its interpretation.

This chapter connected base-R descriptions to sampling variation, mean inference and selected correlation inference. Chapter 6 applies the same reading pattern to counts and group models, introduces a small ggplot2 plotting vocabulary, and examines fitting criteria and procedures across repetitions. Chapters 7–8 extend correlations, comparisons and linear adjustment into regression arguments.

Key terms

These entries use the shared Part One definitions. The first-use explanations above connect them to specific observations and commands; Chapters 2–3 provide the broader descriptive and inferential reference.

Table 12

Computing and simulation terms.

Term and common alternatives	Meaning
Object (R object)	A named value, vector, table, model result, or other item stored in an R session. Assignment with <code><-</code> names an object; the Environment pane lists the current objects.
Vector	A sequence of values of one basic type: numeric, character, or logical. <code>NA</code> marks an unavailable value; it is not zero. A data frame is a table whose columns are vectors of equal length, one row per case.
Function and argument (R function)	A function performs an operation on supplied inputs; an argument names an input or an option, such as <code>na.rm = TRUE</code> . An R argument is distinct from a study argument in the argument-based approach.

Term and common alternatives	Meaning
Package (library)	An installed collection of functions and data, such as stats or graphics, which are supplied with R. Installing an additional package makes it available on the computer; <code>library()</code> attaches it for the session; <code>package::function()</code> names one exported function explicitly.
Script and Quarto source	A saved <code>.R</code> script records commands so an analysis can be rerun; a <code>.qmd</code> source connects prose, mathematics, executable code, tables, figures, and references, and renders to PDF, HTML, or Word.
Simulation and seed (Monte Carlo simulation)	Generating data from a declared process to see what a statistic or procedure does across repetitions. A seed makes the random draws repeatable; it does not make the process true for an actual study.
Monte Carlo standard error	The numerical uncertainty of a simulated probability or mean that comes from using finitely many repetitions; more repetitions shrink it. It is distinct from a study's SE, which depends on the study's sample size.

Table 13

Data-summary terms.

Term and common alternatives	Meaning
Covariance matrix	A square table of variances on the diagonal and covariances off the diagonal for a stated set of variables and observations.
Correlation matrix	A square table of pairwise correlation coefficients for a stated set of variables and observations; its diagonal is 1 for variables with positive variance.

Further reading

R's reference system (R Core Team, 2025) and local Help for `mean`, `sd`, `quantile`, `t.test`, `cor`, `cov2cor` and `cor.test` describe their arguments, defaults and computational conventions. The original papers by Student (1908) and Welch (1947) explain the classical mean procedures developed here. Quarto's documentation (Posit Software, PBC, 2026) gives examples of executable reports, and Peng (2011) connects reproducible computation with broader scientific reporting.

Exercises

Use the stated records and conventions, including each prompt's supplied rounded summaries. Explain the statistical meaning as well as the calculation. Report numerical results to the requested precision.

Exercise 1: Count the right observations (Foundational)

A table contains 40 students with one baseline and one follow-up row each, plus one duplicate follow-up row. There are 81 rows. Explain the difference between a row count and a student count. If four students have missing follow-up scores, give the denominator for the mean observed follow-up score after the documented duplicate is resolved. State what the mean describes.

Exercise 2: Center, spread and a changed observation (Foundational)

Three recorded scores are 50, 60 and 70 points. Calculate the mean, median, sample variance and sample SD. Replace 70 by 100 and calculate the new mean, median and SD. Explain what changed statistically. Use the sample-variance denominator $n - 1$ and report SDs to two decimal places.

Exercise 3: A missing score is not zero (Foundational)

Students A, B and C have baseline scores 50, 60 and 70, and corresponding follow-up scores 55, missing and 78. Calculate the available follow-up mean, the complete-pair mean change and the difference of the separate available means. State each denominator. Explain why equality of two numerical answers would not establish that they used the same student sets.

Exercise 4: Choose the group denominator (Foundational)

A table classifies 19 automatic car models, of which 4 meet a fuel-economy threshold, and 13 manual models, of which 10 meet it. Calculate the percentage meeting the threshold within each transmission group and their manual-minus- automatic percentage-point difference. Then calculate the percentage of threshold-meeting models that are manual. Explain the different denominators.

Exercise 5: Read a relationship before evaluating it (Integrative)

For five paired observations, $x = (-2, -1, 0, 1, 2)$ and $y = (4, 1, 0, 1, 4)$. Calculate covariance and Pearson correlation and describe the scatterplot. A report says, “The correlation is zero, so the variables are unrelated.” First identify its numerical result and reported conclusion, then evaluate whether that conclusion follows. What happens to covariance and correlation if y is multiplied by 10?

Exercise 6: Report paired differences (Integrative)

Four people have condition-1 values 1, 2, 4 and 5 and corresponding condition-2 values 2, 4, 4 and 8 hours. Calculate the mean and sample SD of condition 2 minus condition 1, and verify the difference-of-means identity. Write a descriptive result that names the people, direction and units. Explain why it does not say that every person gained the mean difference.

Exercise 7: Simulate a sampling distribution and read its SE (Integrative)

The waiting-time simulation in Section 3.2 reports, for $n = 30$, a mean of sample means of 9.96 and an empirical SE of 1.79 against a model SE of 1.83.

1. Explain what one histogram bar counts and why the empirical and model SEs differ.
2. Without running R, give the model SE for $n = 120$ and for $n = 480$ and explain the pattern.
3. Say what changes, and what does not, if the number of repetitions is raised from 5,000 to 20,000.
4. In R, change `n_middle <- 30` in the supplied code to 120, rerun from the seed, and report the empirical SE to three decimals.
5. The running mean and the histograms of repeated sample means show different changes as sample size increases. Explain concentration toward a target and increasingly Normal standardized sampling shape, and say whether either makes individual waiting times Normal.

Exercise 8: Name the estimand, estimator, and estimate, then calculate the SE (Integrative)

The special-practice study assigned 60 students to each of two formats; the delayed-assessment means were 27.1 and 24.0 with SDs 6.0 and 6.4.

1. State the estimand, the estimator, and the estimate in one sentence each, and name the repetition behind the uncertainty.
2. Calculate the SE of the mean difference in R from these summaries and report it to four decimals.

3. For the ten sleep differences (SD 1.23 hours), calculate the SE of the mean difference and explain why using the two condition SDs, 1.79 and 2.00, as if the two sets of condition measurements were independent would be a different and larger calculation.
4. A separate simulation estimates a waiting-time variance of 100. Across 5,000 samples of five waits, variances using the $n - 1$ denominator average 101.22 and those using n average 80.98. Explain which rule is unbiased, why 101.22 need not equal 100 in this finite run, and why unbiasedness does not ensure that one estimate is close to the target.

Exercise 9: An interval and its question (Foundational)

Twenty-five independent paired differences have mean 3.0 and SD 5.0. Under the Normal-difference model use $t_{.975,24} = 2.064$. Calculate the estimated SE, 95% interval and statistic for a zero process mean. Then name the appropriate procedures for (a) the same 20 people measured twice and (b) independent groups of 20 and 25 without equal variances. State why the interval is not a range for individual differences.

Exercise 10: A selected relationship (Integrative)

Use the common 32 rows of mpg, hp, and wt. (a) Convert their covariance matrix to a correlation matrix and reconstruct the covariance matrix from the correlations and SDs. (b) Report Pearson and Spearman correlations for mpg/hp and state which association each describes. (c) Compute the Pearson partial correlation controlling linearly for wt, its classical t statistic and correct df, and explain why adjustment alone does not establish a causal effect. (d) For the preselected wt/mpg Pearson test, report the estimate, 95% interval and the conditions supporting the reference.

References

- Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226–1227. <https://doi.org/10.1126/science.1213847>
- Posit Software, PBC. (2026). *Quarto documentation*. Posit Software, PBC. <https://quarto.org/docs/>
- R Core Team. (2025). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Student. (1908). The probable error of a mean. *Biometrika*, 6(1), 1–25. <https://doi.org/10.2307/2331554>
- Welch, B. L. (1947). The generalization of “Student’s” problem when several different population variances are involved. *Biometrika*, 34(1–2), 28–35. <https://doi.org/10.1093/biomet/34.1-2.28>